

TÌM KIẾM (SEARCH)

ptbao@sgu.edu.vn

Cấu trúc dữ liệu mảng

- Định nghĩa: CTDL mảng (array) là CTDL gồm một dãy liên tiếp các phần tử. Các phần tử này có thể được truy cập ngẫu nhiên.
- Các thao tác thông thường trên mảng:
 - ❖ Khởi tạo mảng
 - ❖ Lấy giá trị của phần tử thứ i trong mảng
 - ❖ Đặt giá trị cho phần tử thứ i trong mảng
 - ❖ Xác định kích thước mảng (số phần tử trong mảng)
 - ❖ Thêm một phần tử vào mảng
 - ❖ Huỷ một phần tử ra khỏi mảng
 - ❖ Tìm kiếm một phần tử trong mảng
 - ❖ Sắp xếp mảng

Tìm kiếm

- Phương pháp cơ bản:

1. Tuần tự (tuyến tính – Linear Search)
2. Nhị phân (Binary Search)

- Ý tưởng:

1. Tuần tự: duyệt từ phần tử đầu tiên cho đến phần tử cuối cùng để tìm phần tử thoả điều kiện cần tìm.
2. Nhị phân: phân hoạch làm 02 phần, 01 phần chắc chắn không có, phần còn lại nếu có sẽ tồn tại. Quá trình tìm kiếm sẽ có không gian tìm kiếm thu hẹp, nên việc tìm kiếm sẽ nhanh chóng. Điều kiện các phần tử phải có quan hệ thứ tự nào đó.

Tìm kiếm (tt)

- Thuật giải:

1. Tuần tự:

Thuật giải LSearch(A,n, α)

Input: Mảng A có n phần tử và điều kiện α

Output: Vị trí phần tử cần tìm, hoặc -1 nếu không tìm thấy

for($i \leftarrow 0$ to $n-1$)

if($A[i]$ thoả điều kiện α) return i ;

return -1;

End LSearch;

Tìm kiếm (tt)

- Ưu Khuyết điểm của thuật toán tìm kiếm tuần tự

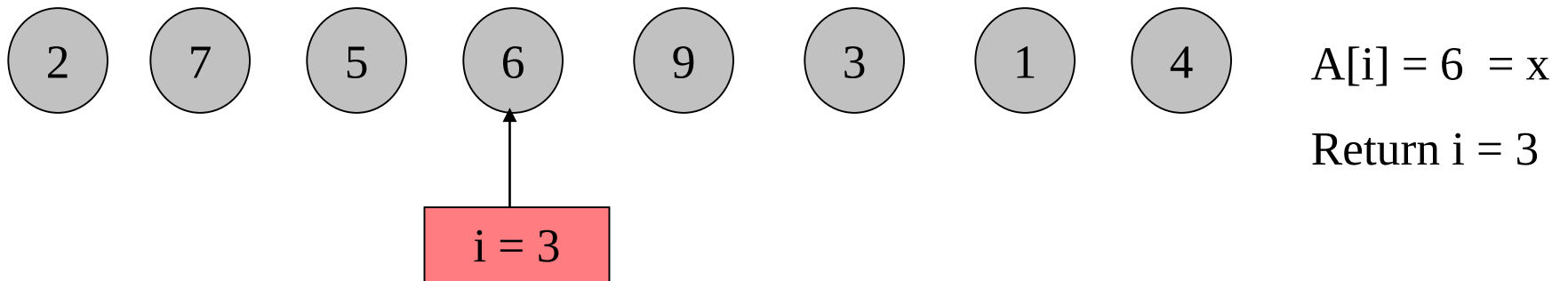
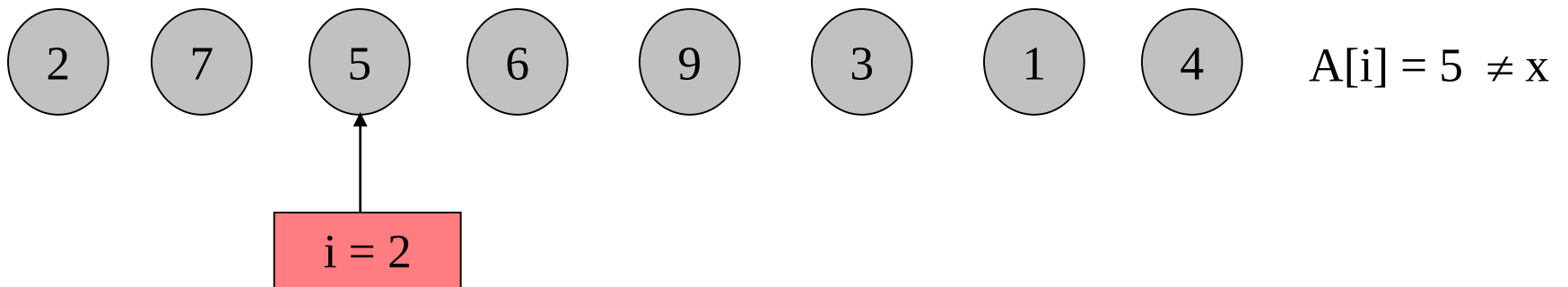
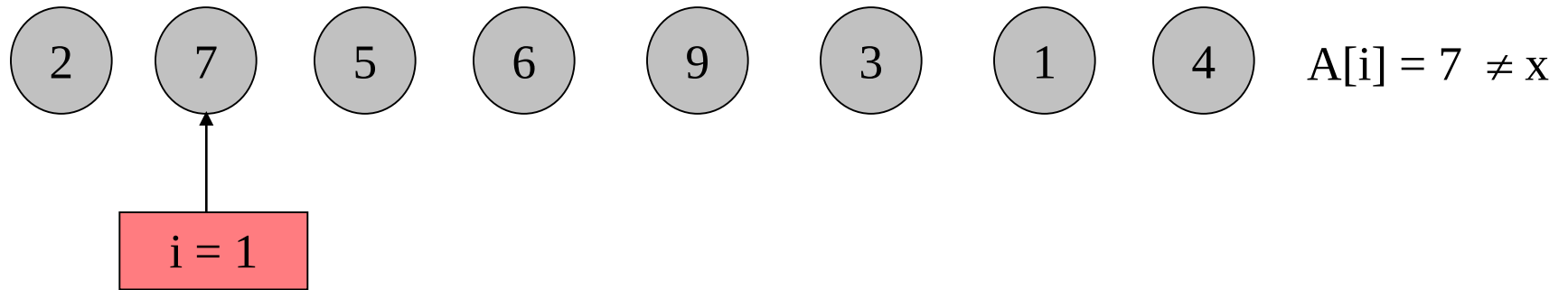
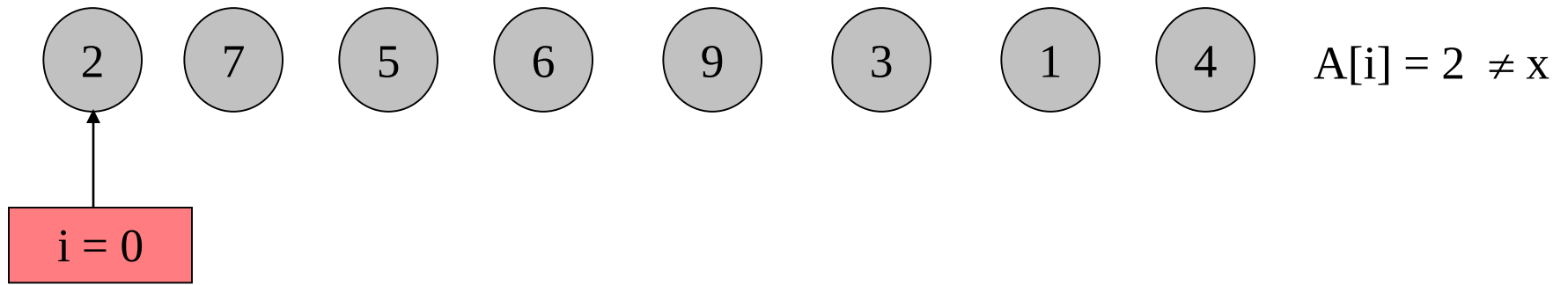
Ưu điểm:

tổng quát, có thể áp dụng trên mọi tiêu chuẩn tìm kiếm cũng như mọi dữ liệu.

Khuyết điểm:

Chi phí cao (tỉ lệ $O(n)$)

Ví dụ: tìm phần tử $x = 6$ trong mảng dưới đây



Tìm kiếm (tt)

2. Nhị phân

Thuật giải BSearch(A,n, α)

Input: Mảng A có n phần tử và điều kiện α

Output: Vị trí phần tử cần tìm, hoặc -1 nếu không tìm thấy

left $\leftarrow$ 0; right $\leftarrow$ n-1;

while(left $\leq$ right) do

 mid $\leftarrow$ (left+right)/2;

 if(A[mid] thoả α) then return mid;

 if(phần tử thoả α ở bên trái) then

 right $\leftarrow$ mid-1;

 else left $\leftarrow$ mid +1;

return -1;

End BSearch;

Tìm kiếm (tt)

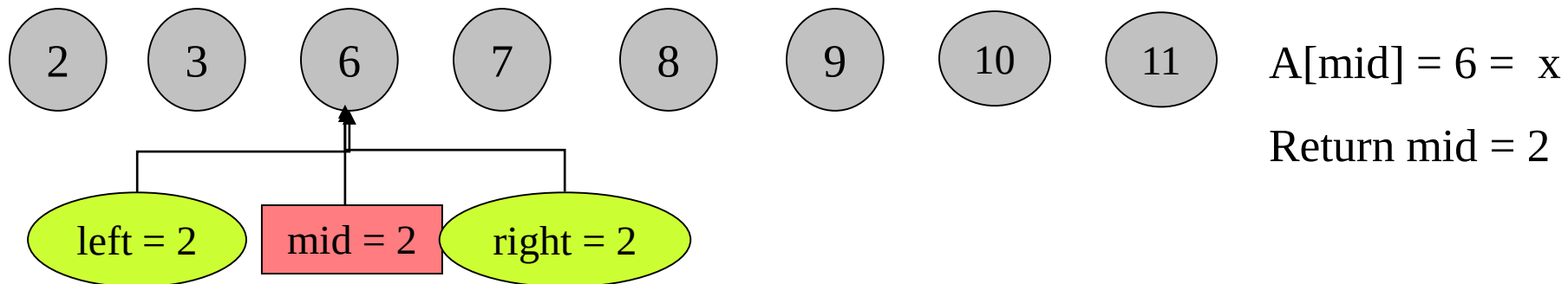
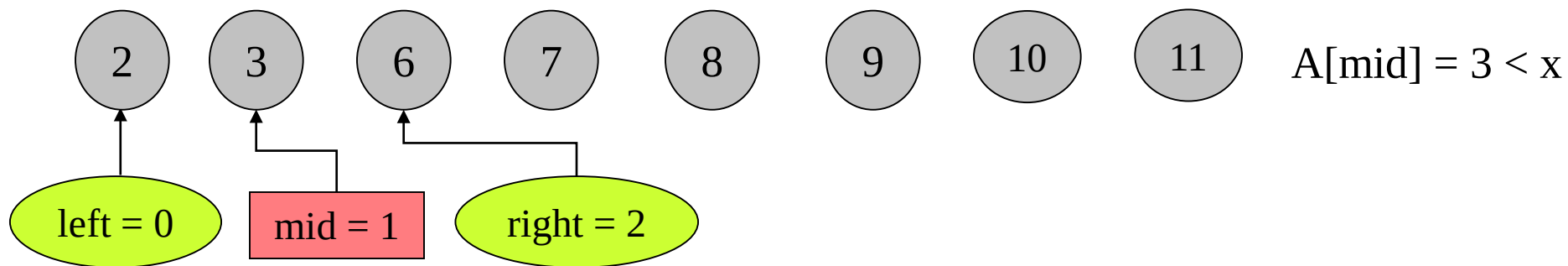
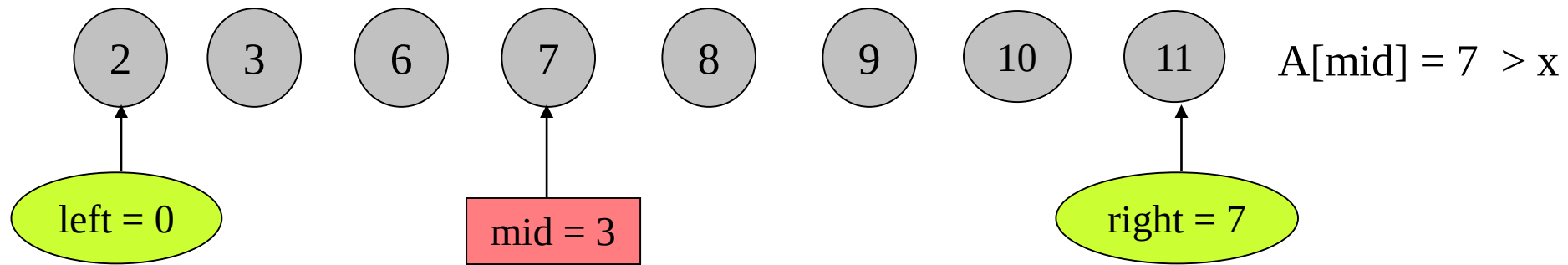
- Ưu Khuyết điểm của thuật toán tìm kiếm nhị phân

Ưu điểm:

Chi phí thấp (tỉ lệ $\log_2(n)$)

Khuyết điểm:

Thuật toán không tổng quát, không thể áp dụng trên mọi tiêu chuẩn tìm kiếm cũng như mọi dữ liệu. Dữ liệu bắt buộc phải có quan hệ thứ tự và việc tìm kiếm phải theo khoá thứ tự này.



SẮP XẾP

(SORT)

1. Đổi chỗ trực tiếp – Interchange Sort
2. Nổi bọt – Bubble Sort
3. Shaker Sort
4. Chèn trực tiếp – Insertion Sort
5. Chèn nhị phân – Binary Insertion Sort
6. Shell Sort
7. Chọn trực tiếp – Selection Sort
8. Heap Sort
9. Quick Sort
10. Merge Sort
11. Radix Sort

Phương pháp đổi chỗ trực tiếp

- Ý tưởng thuật toán:

Khái niệm nghịch thế

Xét một mảng các số $a_0 \ a_1 \ \dots \ a_n$

Nếu có $i < j$ và $a_j < a_i$ thì ta gọi đó là một nghịch thế

Vậy nếu mảng chưa sắp xếp: sẽ có nghịch thế

Nếu mảng đã sắp xếp (có thứ tự): không có nghịch thế
và khi đó a_0 sẽ là phần tử bé nhất rồi đến $a_1 \ a_2 \ \dots$

Đổi chỗ trực tiếp (tt)

Để một mảng được sắp xếp thì số nghịch thế phải giảm dần, để làm được ta sẽ hoán vị cặp phần tử xảy ra nghịch thế.

- Thuật toán:

Thuật toán InterchangeSort(A,n)

Input: Mảng A có n phần tử

Output: Mảng A được sắp xếp

for($i \leftarrow 0$ to $n-2$) do

 for($j \leftarrow i+1$ to $n-1$) do

 if($A[i] > A[j]$) then $A[i] \leftrightarrow A[j]$

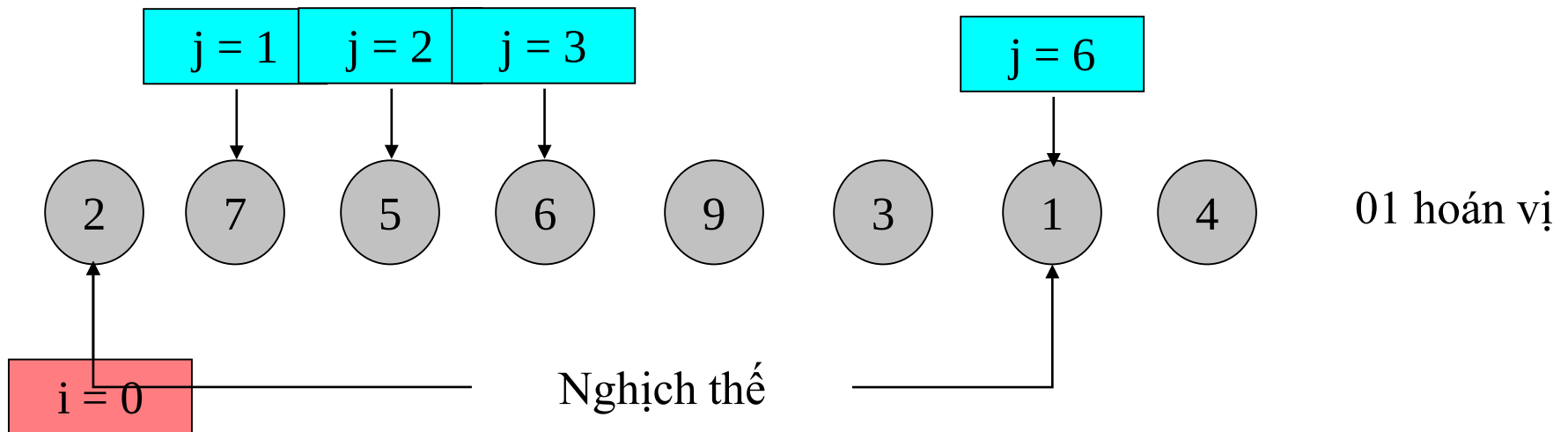
return;

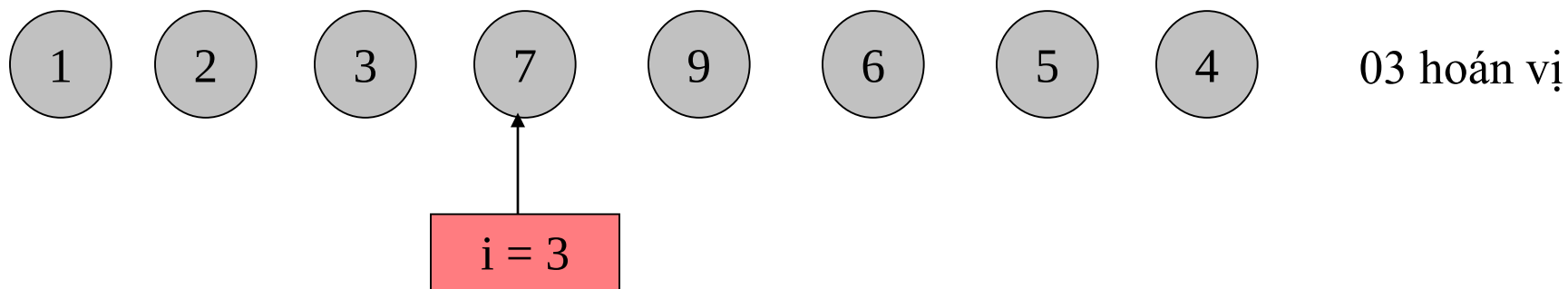
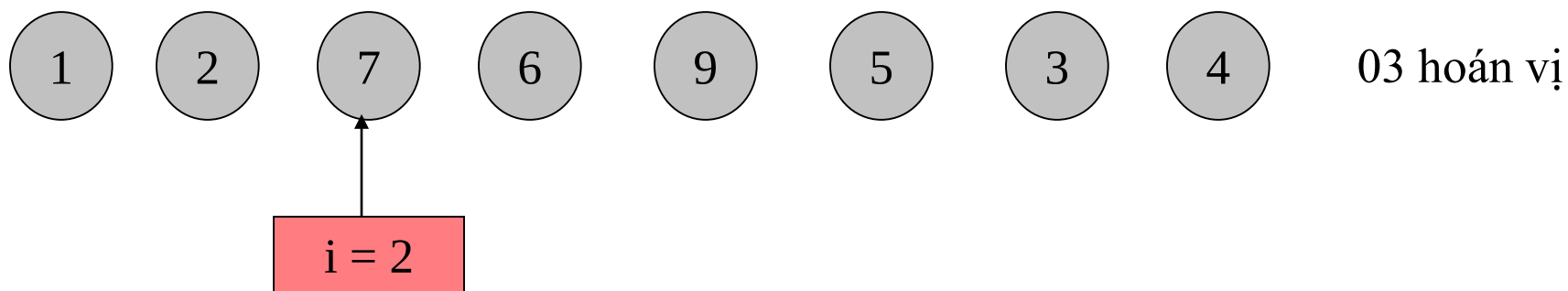
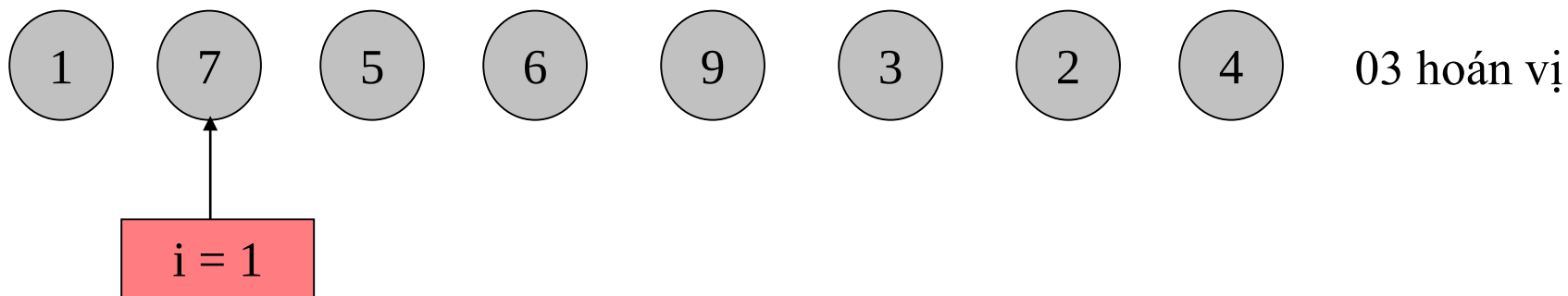
Đổi chỗ trực tiếp (tt)

■ Nhận xét:

- Sau mỗi lần lặp của vòng lặp for i, phần tử $A[i] = \min(A[i], A[i+1], \dots, A[n-1])$
- Thuật toán có độ phức tạp $O(n^2)$

■ Ví dụ:





Phương pháp nổi bọt

- Ý tưởng thuật toán:

Từ một số gợi ý trong thiên nhiên như:

- ✓ Bọt khí
- ✓ Hỗn hợp dầu và nước
- ✓ Thả hỗn hợp gạo và mật cưa vào nước

Ta sẽ đổi chỗ cho 02 phần tử liên tiếp nếu 02 phần tử đó tạo thành 01 nghịch thế.

Phương pháp nổi bọt (tt)

- Thuật toán:

Thuật toán BubbleSort(A,n)

Input: Mảng A có n phần tử

Output: Mảng A được sắp xếp

for($i \leftarrow 0$ to $n-2$) do

 for($j \leftarrow n-1$ to $i+1$) do

 if($A[j-1] > A[j]$) then $A[j-1] \leftrightarrow A[j]$

return;

Phương pháp nổi bọt (tt)

- Nhận xét:

- ✓ Sau mỗi lần lặp của vòng lặp for i, phần tử nhẹ nhất sẽ nổi lên đến mức i; nghĩa là $A[i] = \min(A[i], A[i+1], \dots, A[n-1])$
- ✓ Thuật toán có độ phức tạp $O(n^2)$
- ✓ Thay vì cho vật nhẹ nổi lên ta cũng có thể cho vật nặng chìm xuống

Phương pháp nổi bọt (tt)

Thuật toán BubbleSort1(A,n)

Input: Mảng A có n phần tử

Output: Mảng A được sắp xếp

for($i \leftarrow n-1$ to 1) do

 for($j \leftarrow 0$ to $i-1$) do

 if($A[j] > A[j+1]$) then $A[j] \leftrightarrow A[j+1]$

return;

Phương pháp nổi bọt (tt)

- ✓ Hay đồng thời cho vật nặng chìm xuống và vật nhẹ nổi lên.

Thuật toán BubbleSort2(A,n)

Input: Mảng A có n phần tử

Output: Mảng A được sắp xếp

$up \leftarrow 0$; $down \leftarrow n-1$;

while ($up < down$) do

 for ($j \leftarrow up$ to $down-1$) do

 if($A[j] > A[j+1]$) then $A[j] \leftrightarrow A[j+1]$

$down \leftarrow down - 1$;

 for($j \leftarrow down$ to $up+1$) do

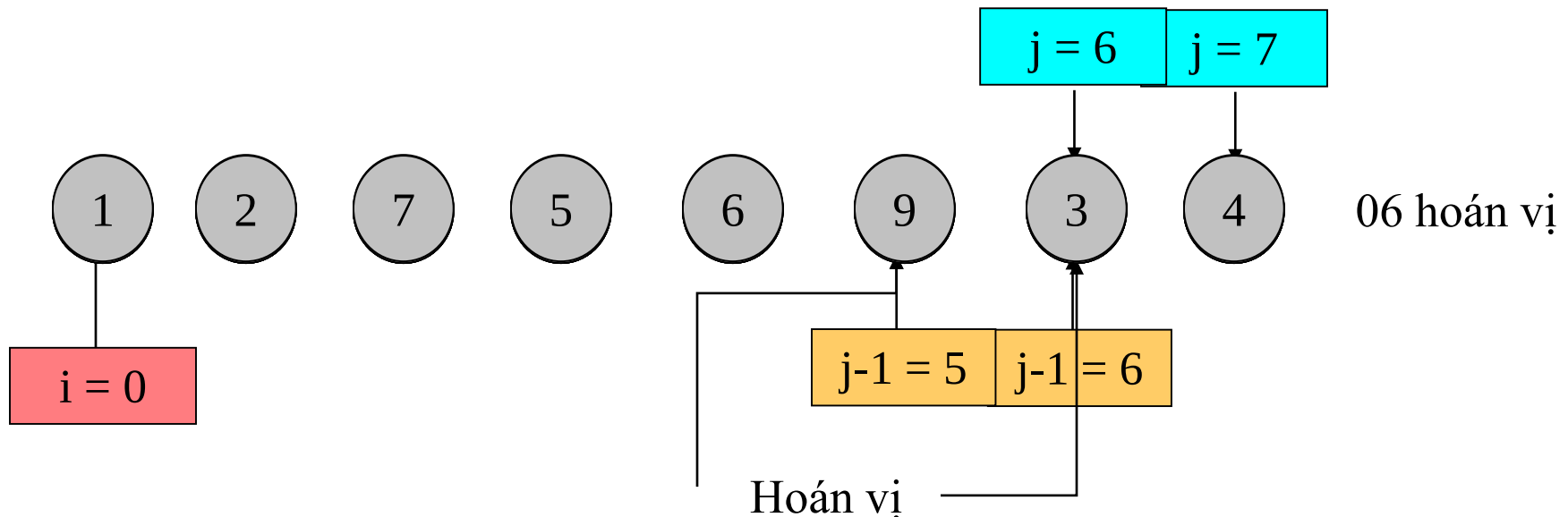
 if($A[j-1] > A[j]$) then $A[j-1] \leftrightarrow A[j]$

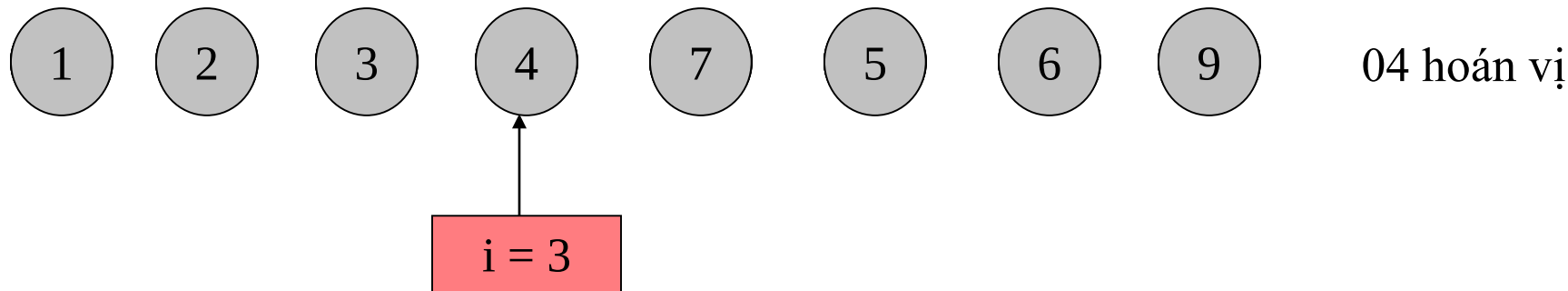
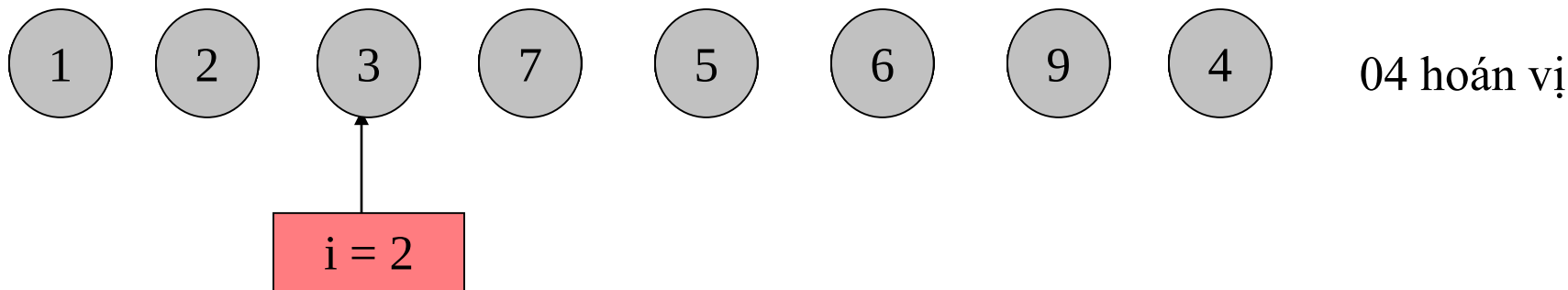
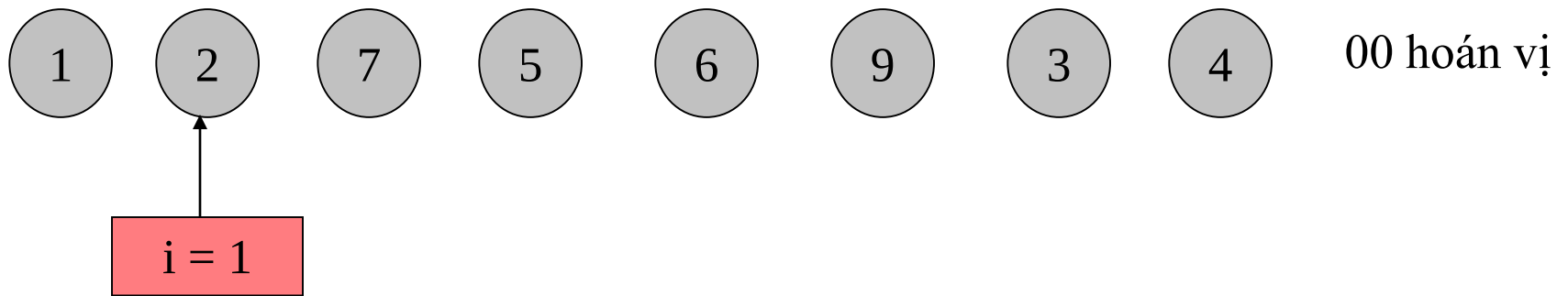
$up \leftarrow up + 1$;

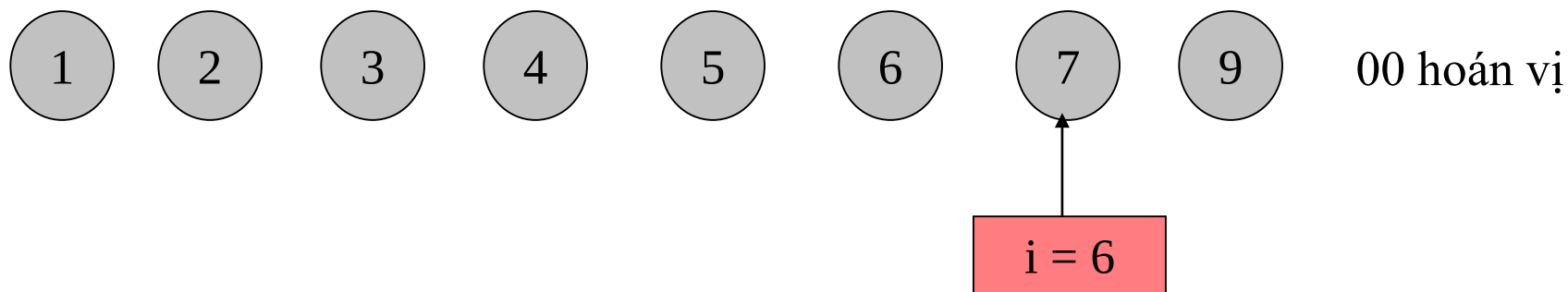
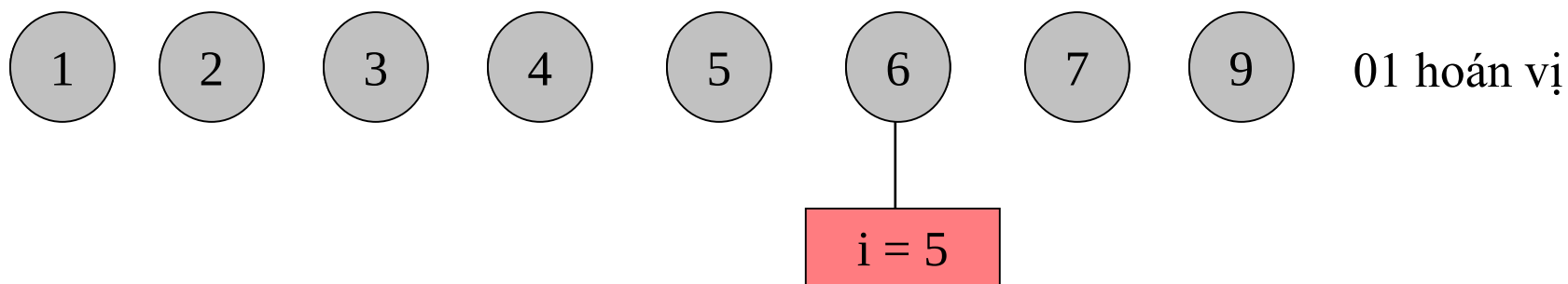
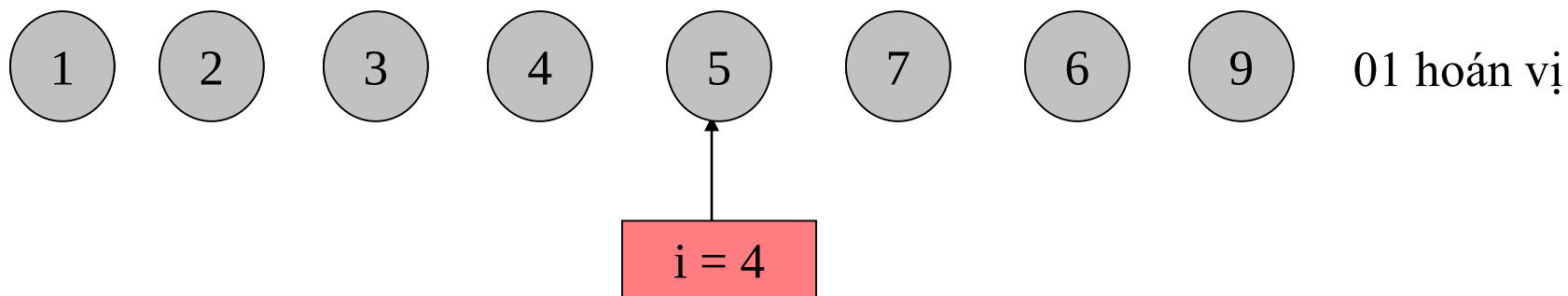
return;

Phương pháp nổi bọt (tt)

- ✓ Dù thuật toán vẫn có độ phức tạp $O(n^2)$ nhưng chi phí sẽ giảm nhiều, vì ta đã lợi dụng tính đối ngẫu.
- Ví dụ 1 (nhẹ nổi lên)

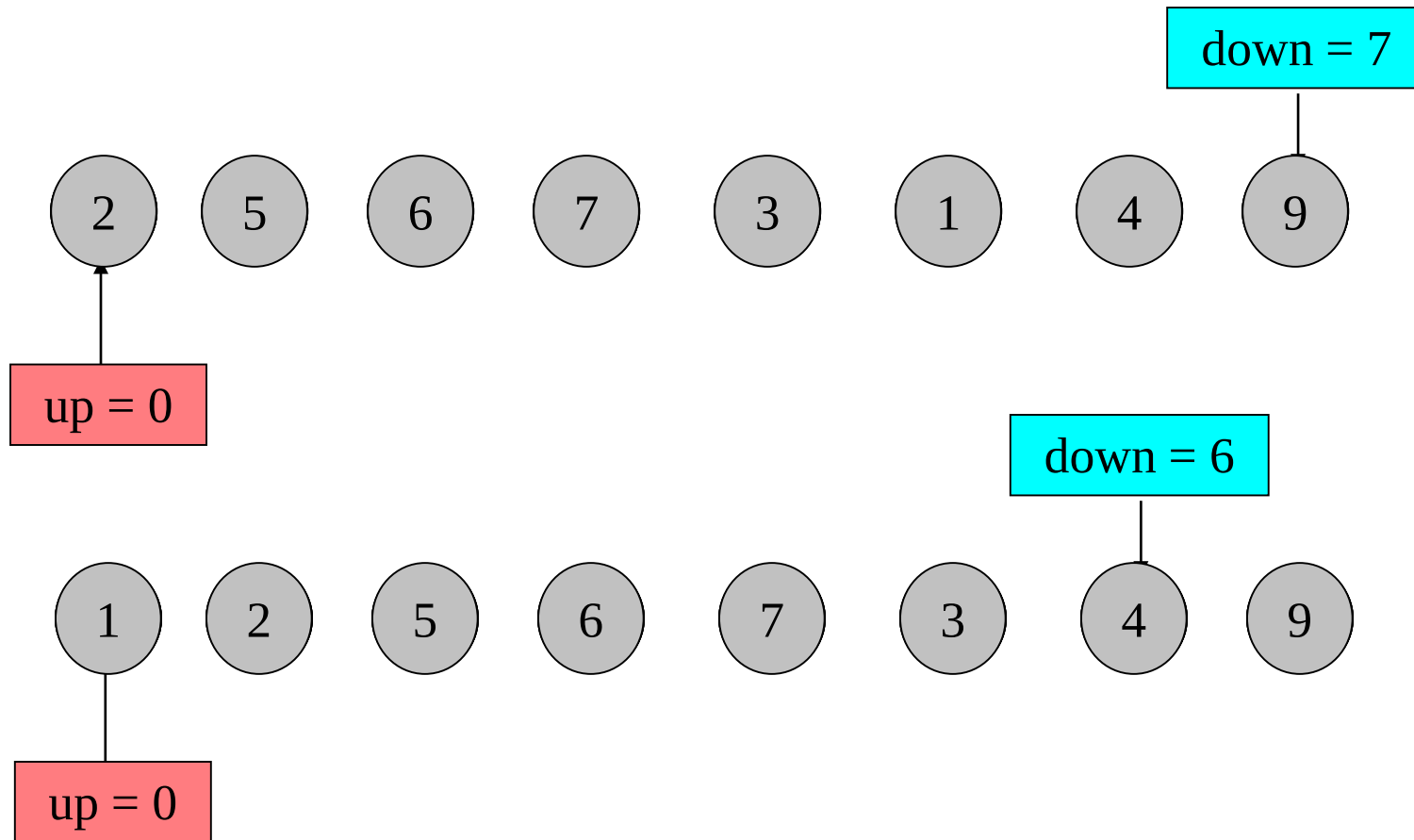


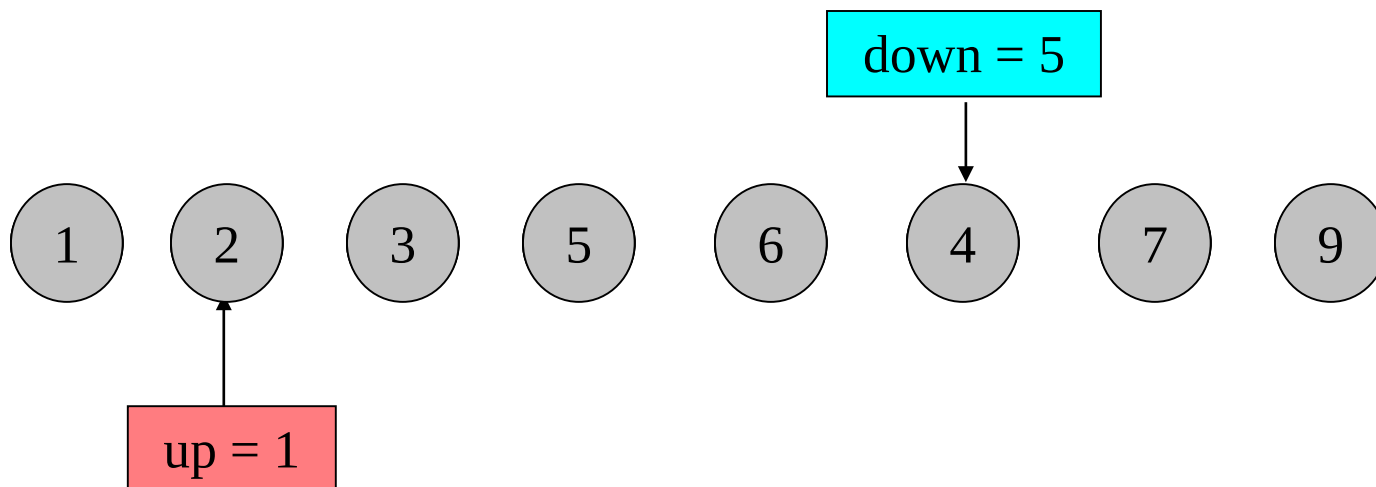
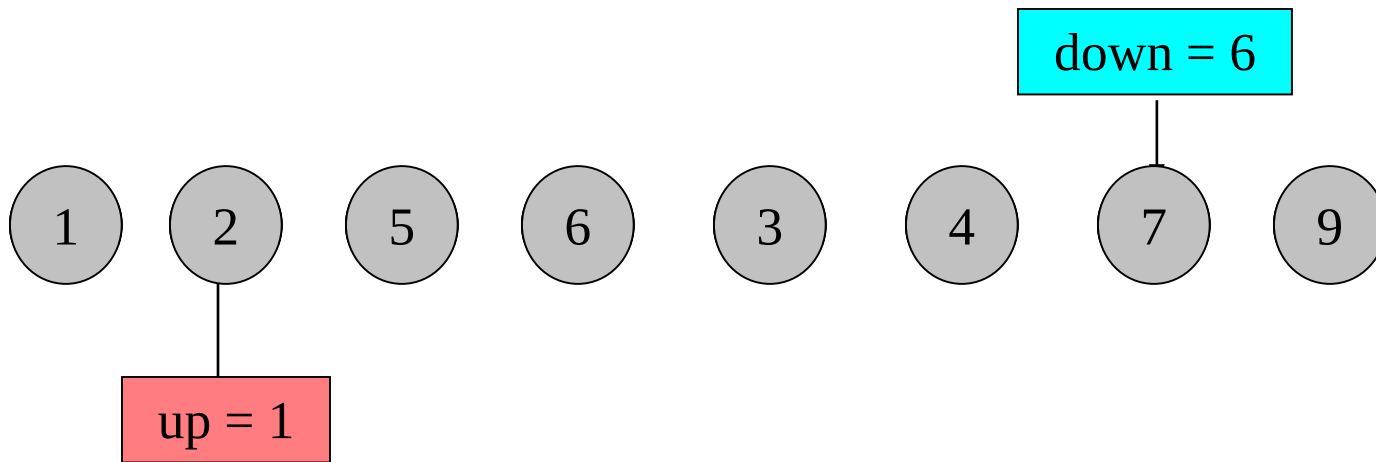


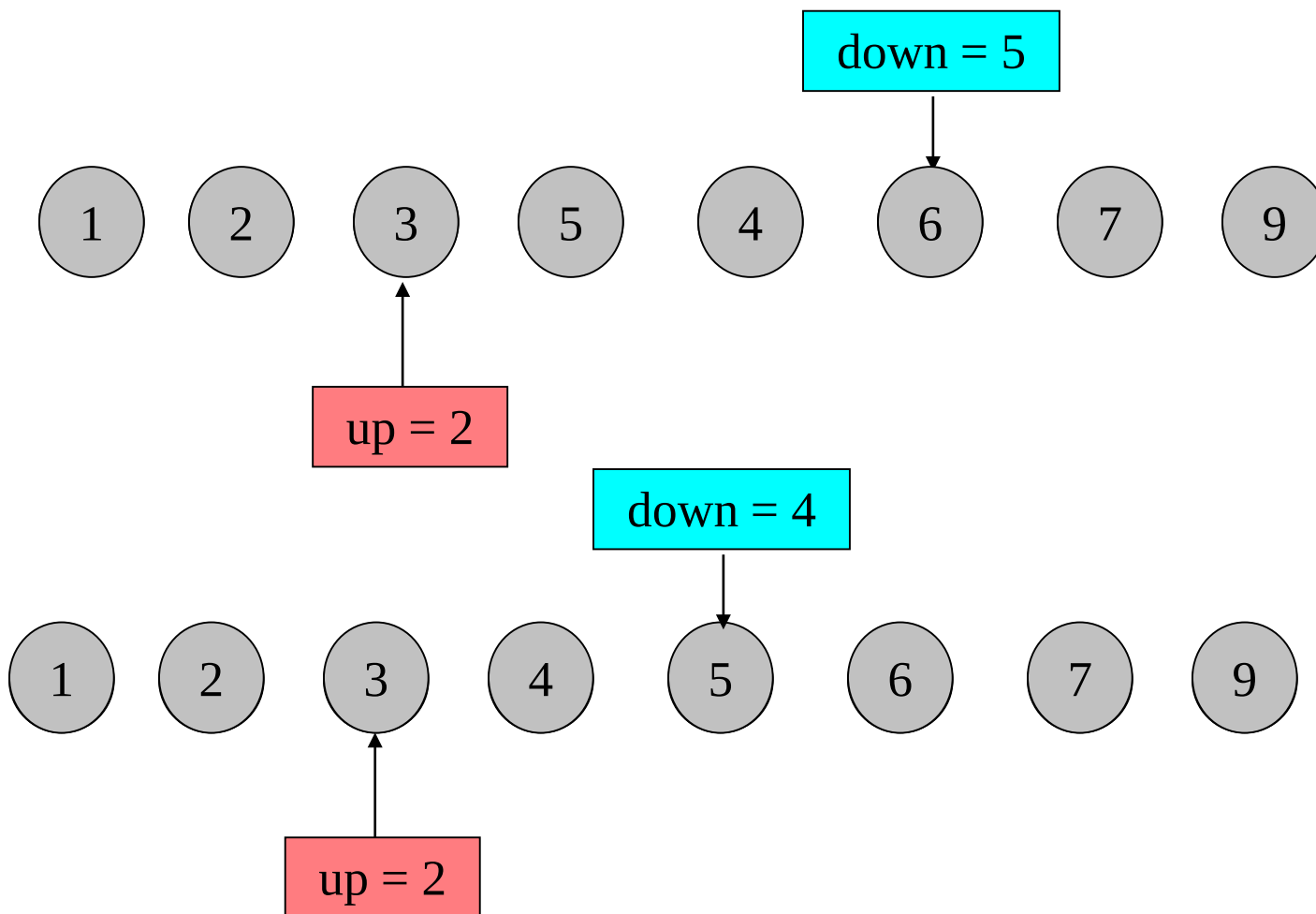


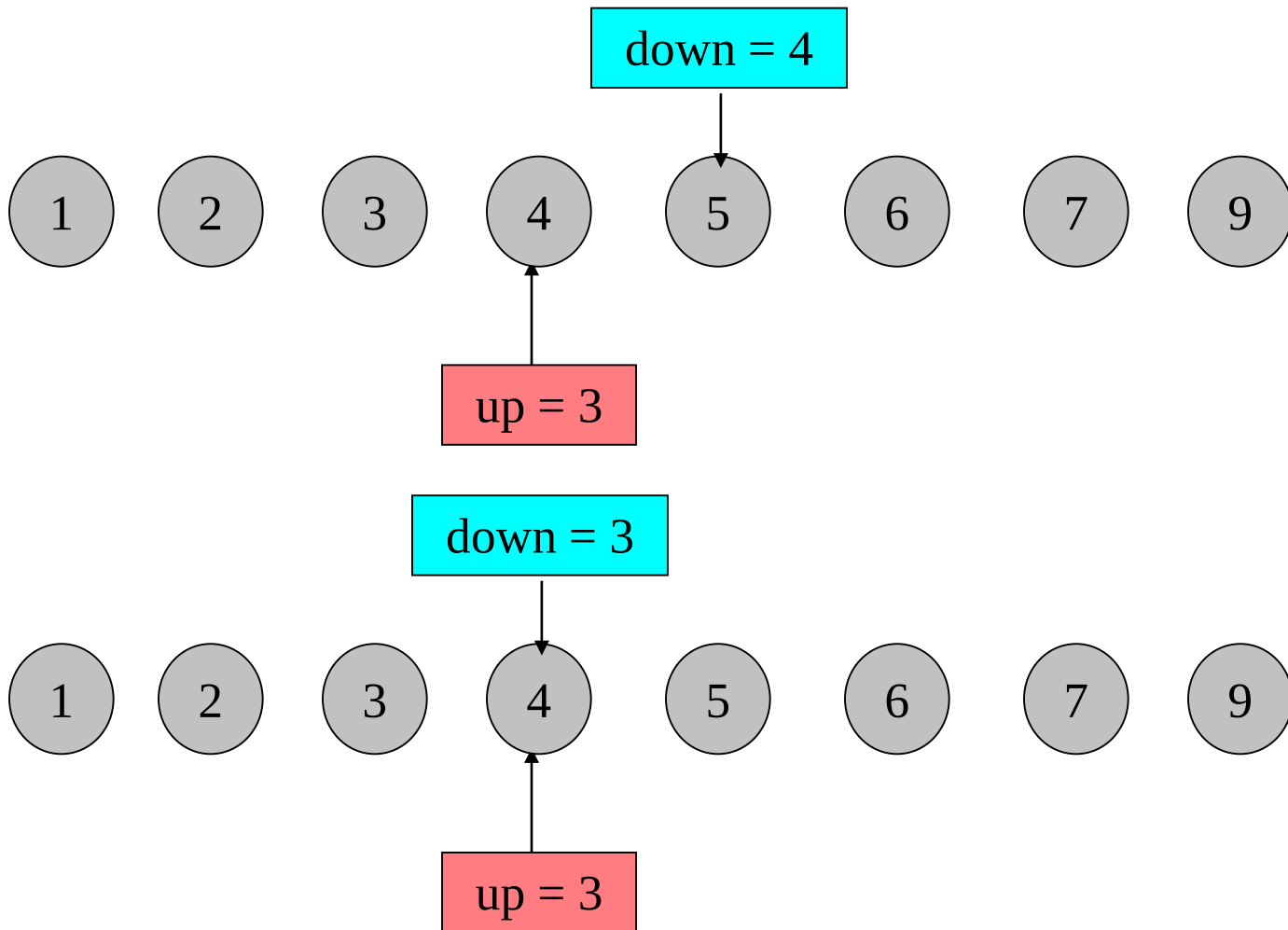
Kết thúc → Mảng có thứ tự

■ Ví dụ 2 (nhẹ nổi lên và nặng chìm xuống)









Kết thúc → Mảng có thứ tự

Phương pháp Shaker Sort

- Ý tưởng thuật toán:

Phương pháp Bubble Sort có một nhược điểm rất lớn là khi dữ liệu có thể đã được sắp xếp cục bộ thì phương pháp Bubble Sort vẫn phải làm tuần tự mà không lợi dụng được tính chất này. Do đó Shaker Sort là phương pháp cải tiến của Bubble Sort để giảm số lần lặp.

Phương pháp Shaker Sort (tt)

■ Thuật toán:

Thuật toán ShakerSort(A,n)

Input: Mảng A có n phần tử

Output: Mảng A được sắp xếp

$up \leftarrow 0$; $down \leftarrow n-1$; $hv \leftarrow 0$;

while ($up < down$) do

 for ($j \leftarrow up$ to $down-1$) do

 if($A[j] > A[j+1]$) then

$A[j] \leftrightarrow A[j+1]$;

$hv \leftarrow j$;

$down \leftarrow hv$;

 for($j \leftarrow down$ to $up+1$) do

 if($A[j-1] > A[j]$) then

$A[j-1] \leftrightarrow A[j]$;

$hv \leftarrow j$;

$up \leftarrow hv$;

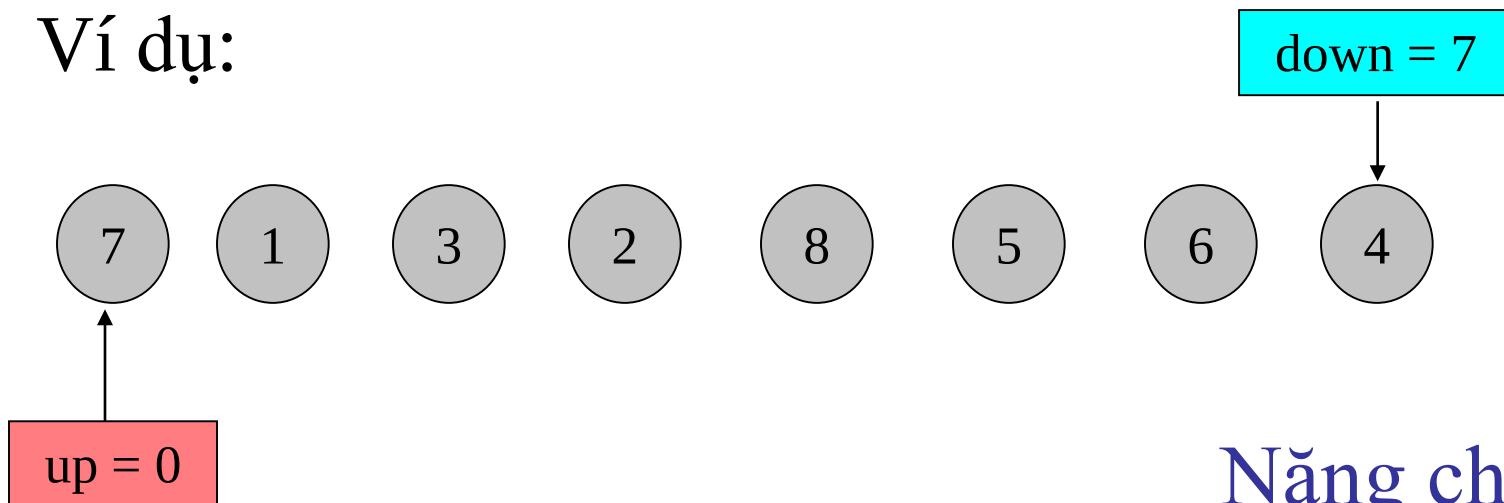
return;

Phương pháp Shaker Sort (tt)

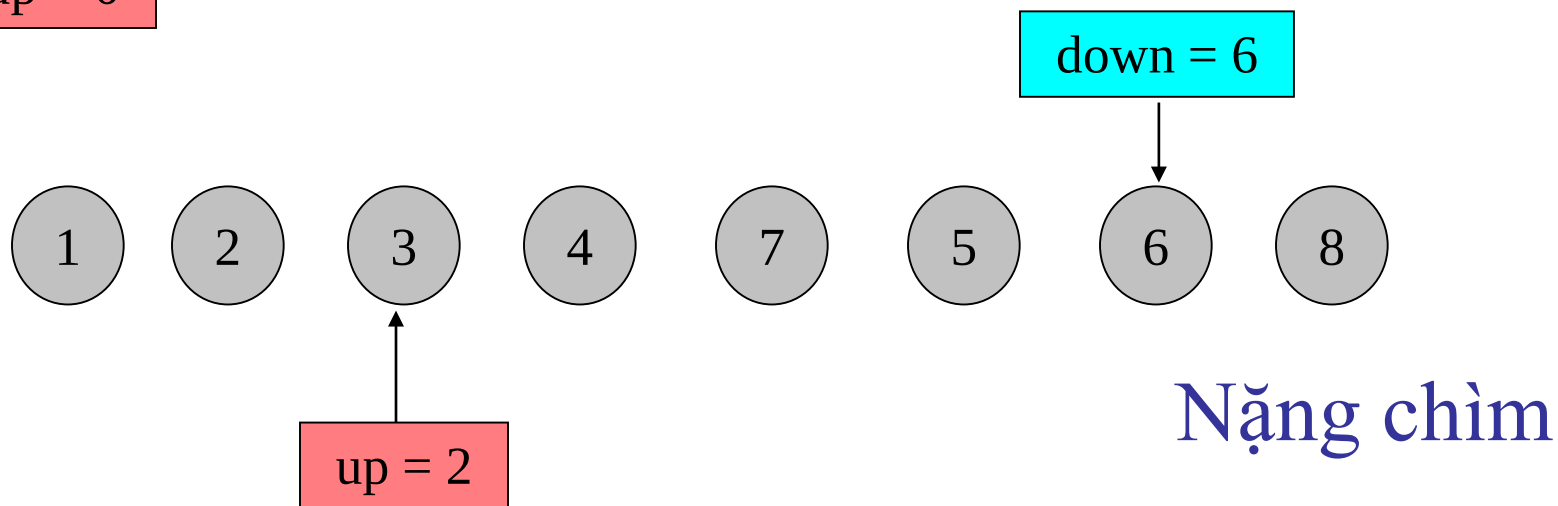
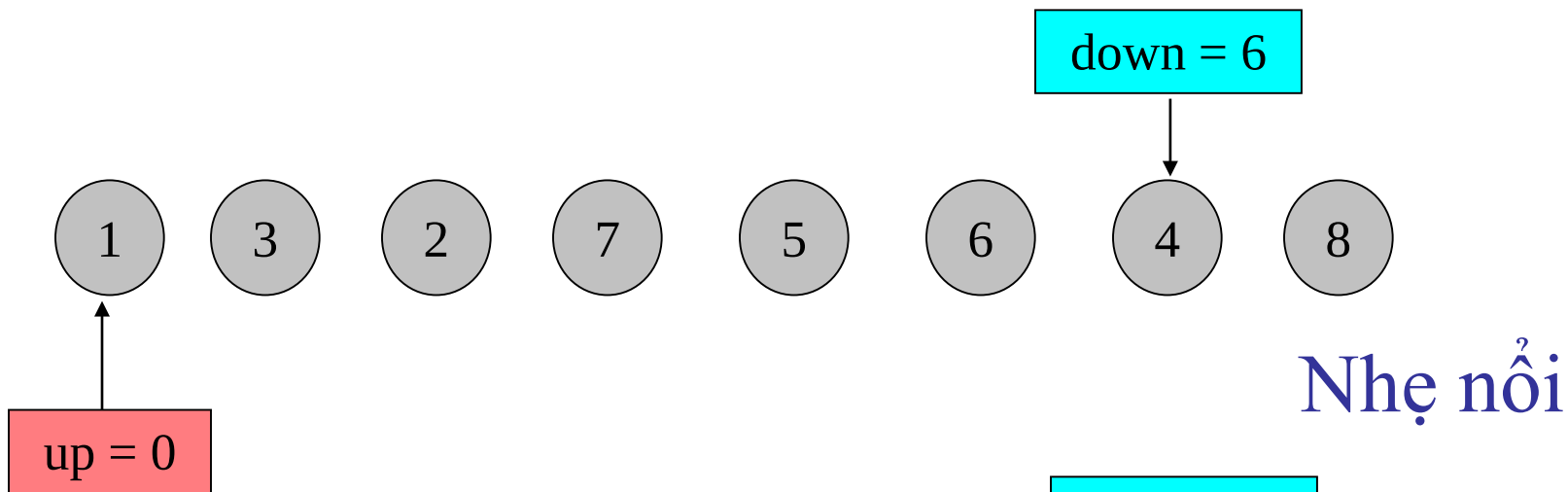
■ Nhận xét:

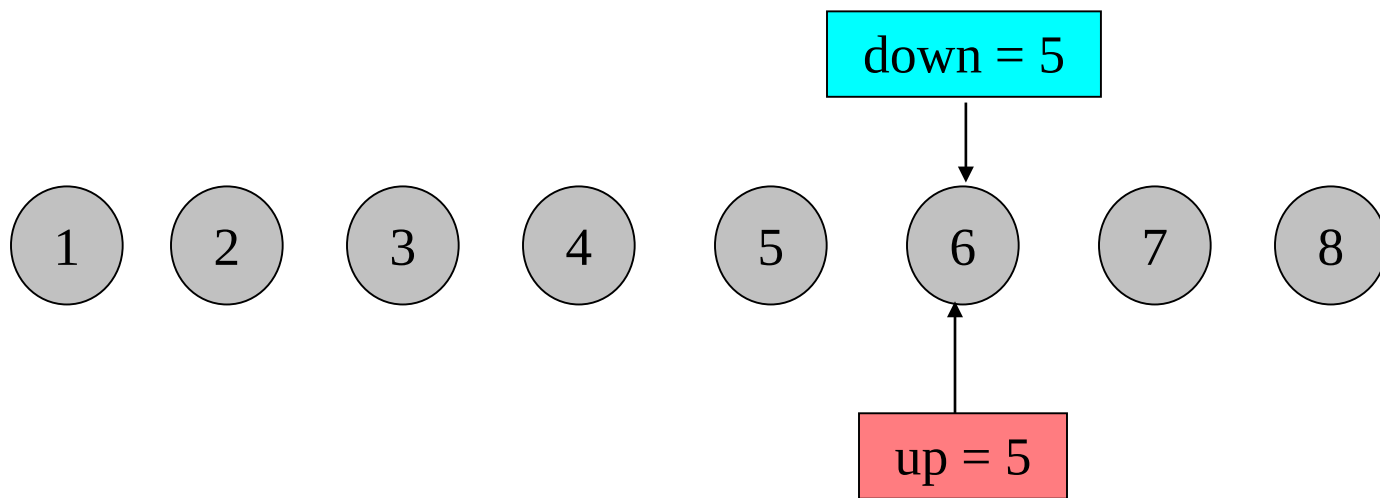
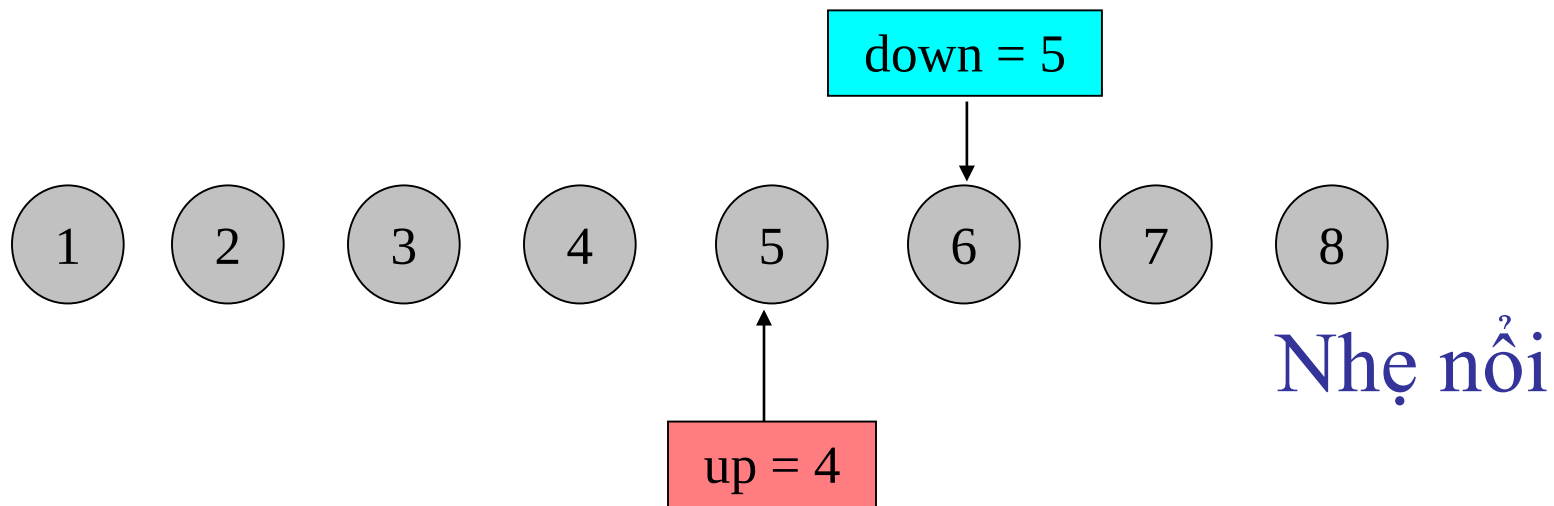
- ✓ Thuật toán có độ phức tạp $O(n^2)$
- ✓ Tuy nhiên qua thực nghiệm, nếu dữ liệu có thứ tự cục bộ nhiều thì Shaker Sort sẽ nhanh hơn rất nhiều so với Bubble Sort.

■ Ví dụ:



Nặng chìm





Kết thúc → Mảng có thứ tự

Phương pháp chèn trực tiếp

- Ý tưởng thuật toán:

Xét mảng A có các số $a_0 \ a_1 \ \dots \ a_{i-1} \ a_i \ a_n$

Giả sử tại thời điểm thứ i đã có i phần tử đầu tiên của mảng đã có thứ tự $a_0 \leq a_1 \leq \dots \leq a_{i-1}$

Nếu ta tìm được vị trí k ($0 \leq k \leq i$) sao cho $a_{k-1} \leq a_i \leq a_k$ ta sẽ chèn a_i vào giữa a_{k-1} và a_k ta sẽ nhận được trạng thái mới của mảng A trong đó $i+1$ phần tử đầu tiên có thứ tự tăng.

Phương pháp chèn trực tiếp (tt)

- Thuật toán:

Thuật toán InsertionSort(A,n)

Input: Mảng A có n phần tử

Output: Mảng A có thứ tự

for($i \leftarrow 1$ to $n-1$) do

$x \leftarrow A[i];$

$j \leftarrow i-1;$

 while ($x < A[j] \ \&\& \ j \geq 0$) do

$A[j+1] \leftarrow A[j];$

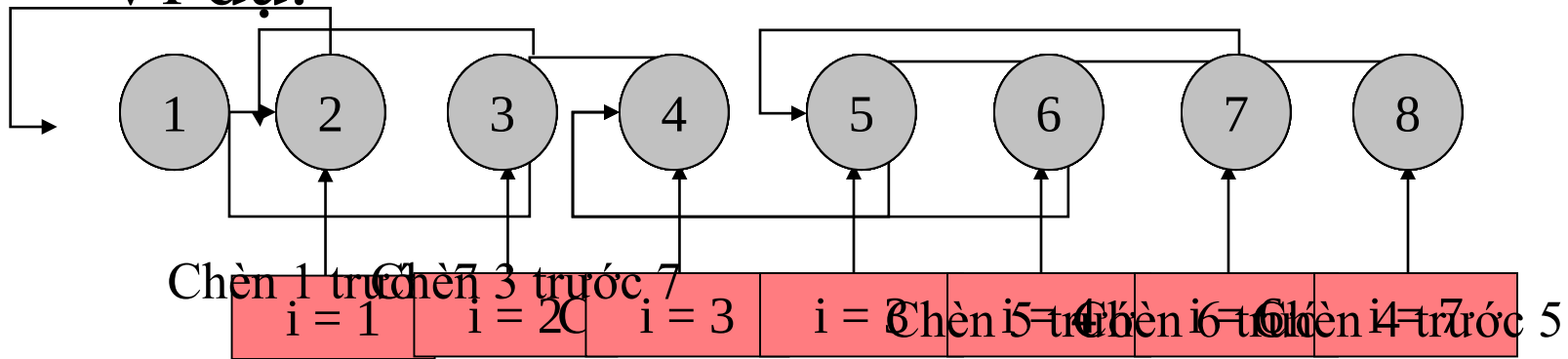
$j \leftarrow j-1;$

$A[j+1] \leftarrow x;$

return;

Phương pháp chèn trực tiếp (tt)

- Ví dụ:



Kết thúc → Mảng có thứ tự

Phương pháp chèn trực tiếp (tt)

- Nhận xét:
 - ✓ Thuật toán có độ phức tạp $O(n^2)$
 - ✓ Dữ liệu càng ít nghịch thế, thuật toán chạy càng nhanh.

Phương pháp chèn nhị phân

- Ý tưởng thuật toán:

Trong thuật toán chèn trực tiếp, khi tìm vị trí k thỏa $a_{k-1} \leq a_i \leq a_k$, phương pháp tìm vị trí là *tuần tự*, như thế ta có thể dùng phương pháp tìm kiếm *nhị phân* để cải tiến do $a_0 \leq a_1 \leq \dots \leq a_{i-1}$

Phương pháp chèn nhị phân (tt)

■ Thuật toán:

Thuật toán BinaryInsertionSort(A,n)

Input: Mảng A có n phần tử

Output: Mảng A có thứ tự

for($i \leftarrow 1$ to $n-1$) do

$x \leftarrow A[i];$

$k \leftarrow \text{BinarySearch}(A,i,x);$

 for($j \leftarrow i$ to $k+1$) do

$A[j] \leftarrow A[j-1];$

$A[k] \leftarrow x;$

return;

Phương pháp chèn nhị phân (tt)

- Nhận xét:

- ✓ Thuật toán có độ phức tạp $O(n^2)$ với thao tác gán nhưng có độ phức tạp $O(n \log_2 n)$ với thao tác so sánh.
- ✓ Nhưng phương pháp chèn nhị phân sẽ nhanh hơn đáng kể khi dữ liệu rất nhiều và đặc biệt khi chi phí so sánh giữa 02 phần tử là cao, ví dụ: string, cấu trúc,

Phương pháp Shell Sort

- Ý tưởng thuật toán:

Khi dữ liệu càng ít hỗn độn (ít nghịch thế) thì thuật toán chèn trực tiếp càng chạy nhanh, ta có thuật toán chèn nhị phân là một cải tiến. Một hướng tiếp cận khác là làm sao giảm bớt số nghịch thế trước khi thực sự sắp xếp. Shell sort là cải tiến của chèn trực tiếp theo hướng tiếp cận này.

Ta sẽ phân hoạch mảng thành nhiều dãy con đan xen nhau, sắp xếp các dãy con này trước khi ta dùng phương pháp chèn trực tiếp để sắp xếp mảng.

Phương pháp Shell Sort (tt)

Chúng ta phải phân hoạch các bước nhảy như sau $h_0 > h_1 > \dots > h_k$, và ở bước cuối cùng phải là 1. Tại các bước ta sẽ dùng thuật toán chèn trực tiếp để sắp xếp các dãy con.

Phương pháp Shell Sort (tt)

- Thuật toán:

Thuật toán ShellSort(A,n)

Input: Mảng A có n phần tử

Output: Mảng A có thứ tự

Chọn $h[0], h[1], \dots, h[k-1] = 1;$

for($i \leftarrow 0$ to $k-1$) do

 Phân hoạch mảng A thành t dãy có chiều dài $h[i]$

 for($j \leftarrow 0$ to $t-1$) do

 InsertionSort(dãy con đã phân hoạch);

return;

Phương pháp Shell Sort (tt)

Nhưng trong cài đặt, chúng ta sẽ đồng thời sắp xếp từng dãy con.

- Thuật toán:

Thuật toán ShellSort(A,n)

Input: Mảng A có n phần tử

Output: Mảng A có thứ tự

Chọn $h[0], h[1], \dots, h[k-1] = 1$;

for($p \leftarrow 0$ to $k-1$) do

 for($i \leftarrow h[p]$ to $n-1$) do

$x \leftarrow A[i]; j \leftarrow i$;

 while($x < A[j-h[p]] \ \&\& \ j \geq h[p]$) do

$A[j] \leftarrow A[j-h[p]]$;

$j \leftarrow j - h[p]$;

$A[j] \leftarrow x$;

return;

Phương pháp Shell Sort (tt)

■ Nhận xét:

- ✓ Thuật toán có độ phức tạp nhỏ hơn $O(n^{3/2})$ Tuy nhiên Shell Sort hiệu quả hơn nhiều so với chèn trực tiếp.
- ✓ Tính hiệu quả sẽ phụ thuộc rất nhiều vào cách chọn các bước nhảy. Tuy nhiên, hiện nay không có tiêu chuẩn tổng quát để chọn các bước nhảy, mà thông qua kinh nghiệm ta sẽ có cách chọn. Nhưng chọn như thế nào đi nữa, ta luôn làm sao chọn các bước nhảy mà phân hoạch mảng thành các dãy ít trùng lặp càng tốt.

Phương pháp Shell Sort (tt)

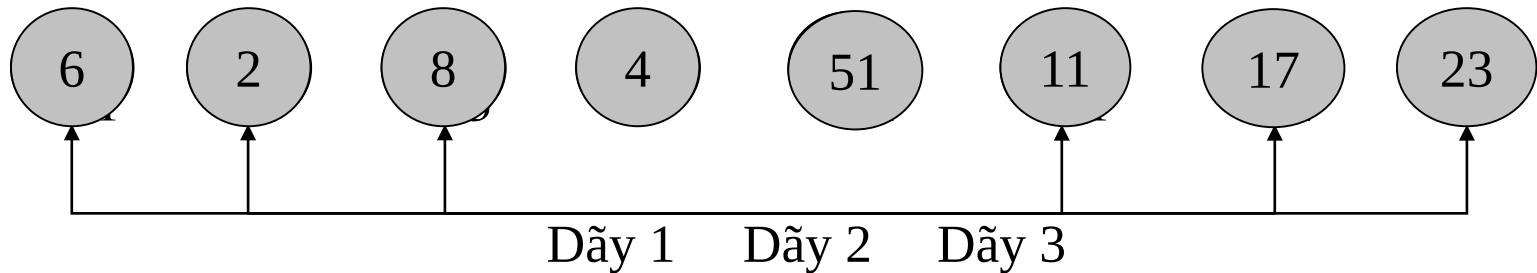
Một số cách chọn:

- h là dãy các số nguyên tố giảm dần đến 1: 13, 11, 7, 5, 3, 1. Đôi khi không nhất thiết các số nguyên tố này liên tục, ví dụ: 13, 5, 3, 1.
- h có dạng $3i+1$: 364, 121, 40, 13, 4, 1
- Dãy fibonacci: 34, 21, 13, 8, 5, 3, 2, 1
- Dãy 4193, 1037, 281, 77, 23, 8, 1
- Dãy 1968, 861, 336, 112, 48, 21, 7, 3, 1
- Dãy 511, 255, 127, 63, 31, 15, 7, 3, 1

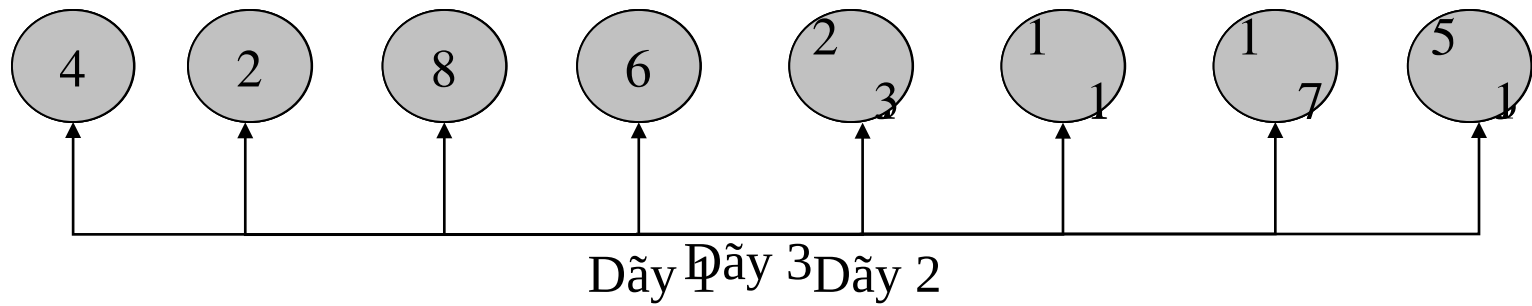
Phương pháp Shell Sort (tt)

- Ví dụ: xét dãy 5,3,1

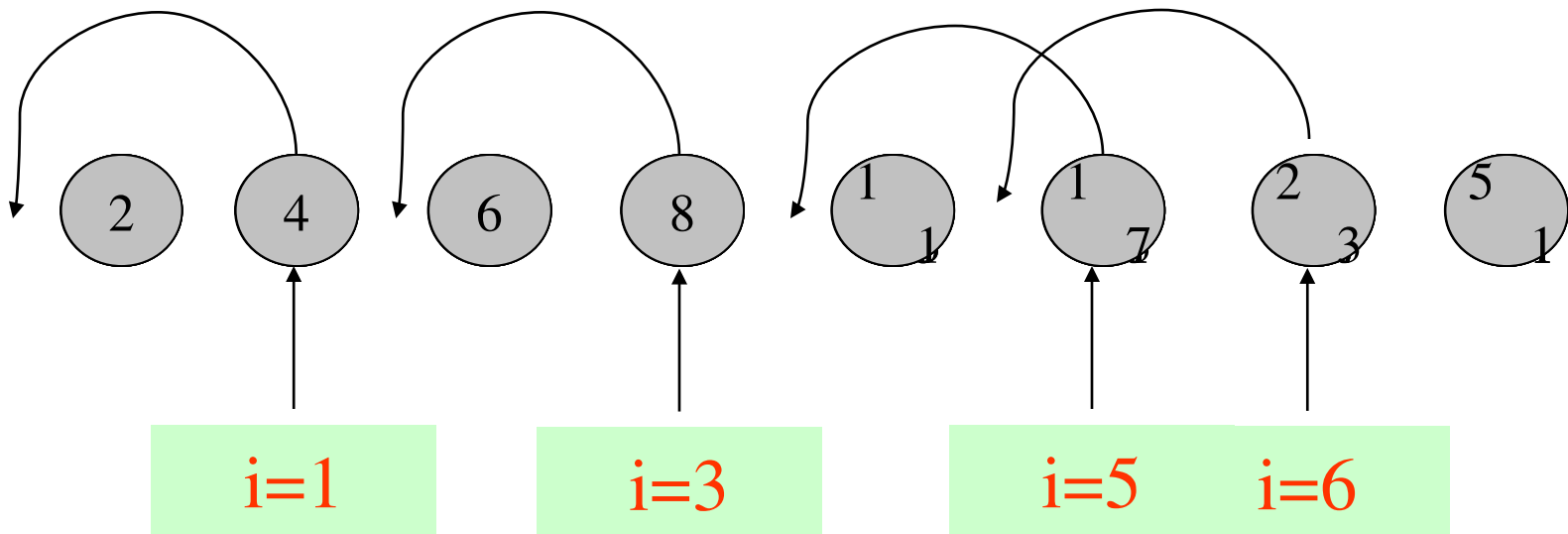
Với $h[0] = 5$



Với $h[1] = 3$



Với $h[2] = 1$



Kết thúc → Mảng có thứ tự

Phương pháp chọn trực tiếp

- Ý tưởng thuật toán:

Ta thấy rằng nếu mảng có thứ tự thì $A[i]$ luôn là phần tử bé nhất của tập $(A[i], A[i+1], \dots, A[n])$. Vậy một cách đơn giản ta đặt phần tử nhỏ nhất vào đầu mảng, tìm phần tử nhỏ kế tiếp trong tập còn lại rồi đặt vào vị trí kế tiếp, cứ như vậy cho đến hết ta có mảng được sắp có thứ tự.

Phương pháp chọn trực tiếp (tt)

- Thuật toán:

Thuật toán SelectionSort(A,n)

Input: Mảng A có n phần tử

Output: Mảng A có thứ tự

for($i \leftarrow 0$ to $n-2$) do

$k \leftarrow i$;

 for($j \leftarrow i+1$ to $n-1$) do

 if($A[k] > A[j]$) then $k \leftarrow j$;

$A[k] \leftrightarrow A[i]$;

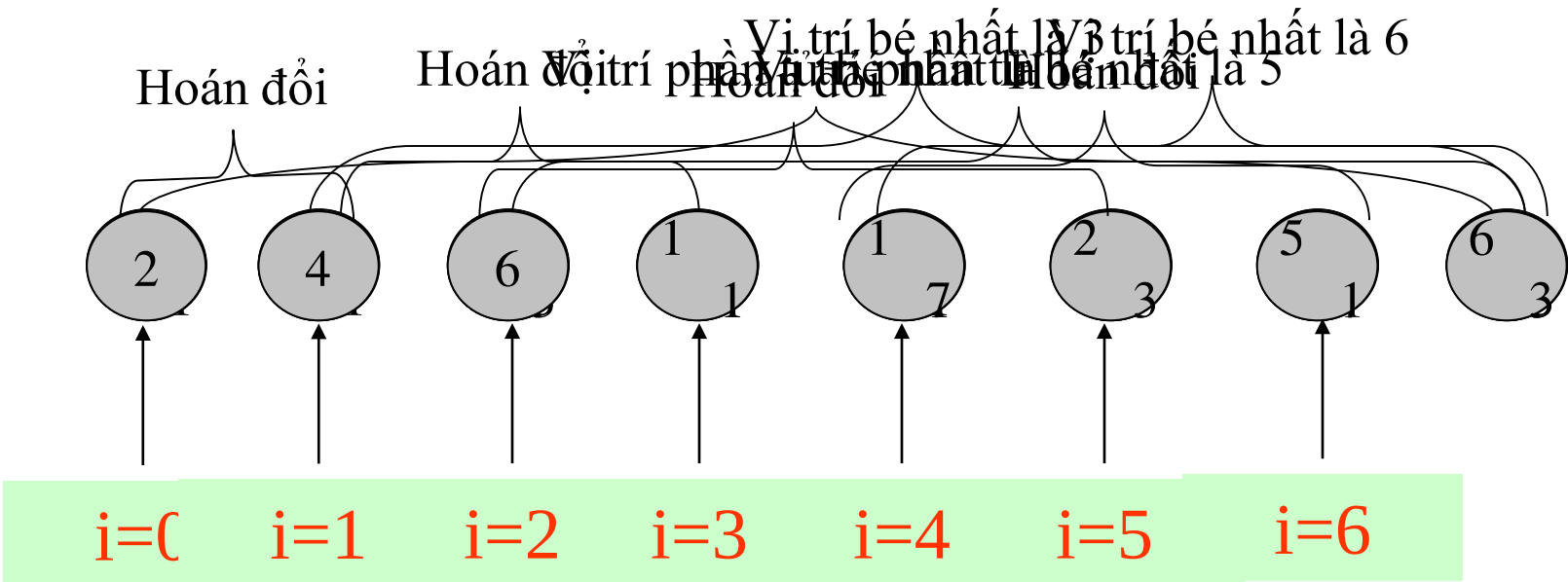
return;

Phương pháp chọn trực tiếp (tt)

- Nhận xét:
 - ✓ Thuật toán có độ phức tạp $O(n^2)$.

Phương pháp chọn trực tiếp (tt)

- Ví dụ:



Kết thúc $\rightarrow$ Mảng có thứ tự

Phương pháp Heap Sort

- Ý tưởng thuật toán:

Ta thấy rằng trong thuật toán chọn trực tiếp, cứ mỗi vòng lặp ta phải tìm phần tử nhỏ nhất trong số các phần tử còn lại. Trong quá trình tìm kiếm ta không tận dụng được các thông tin ở các lần tìm trước đó.

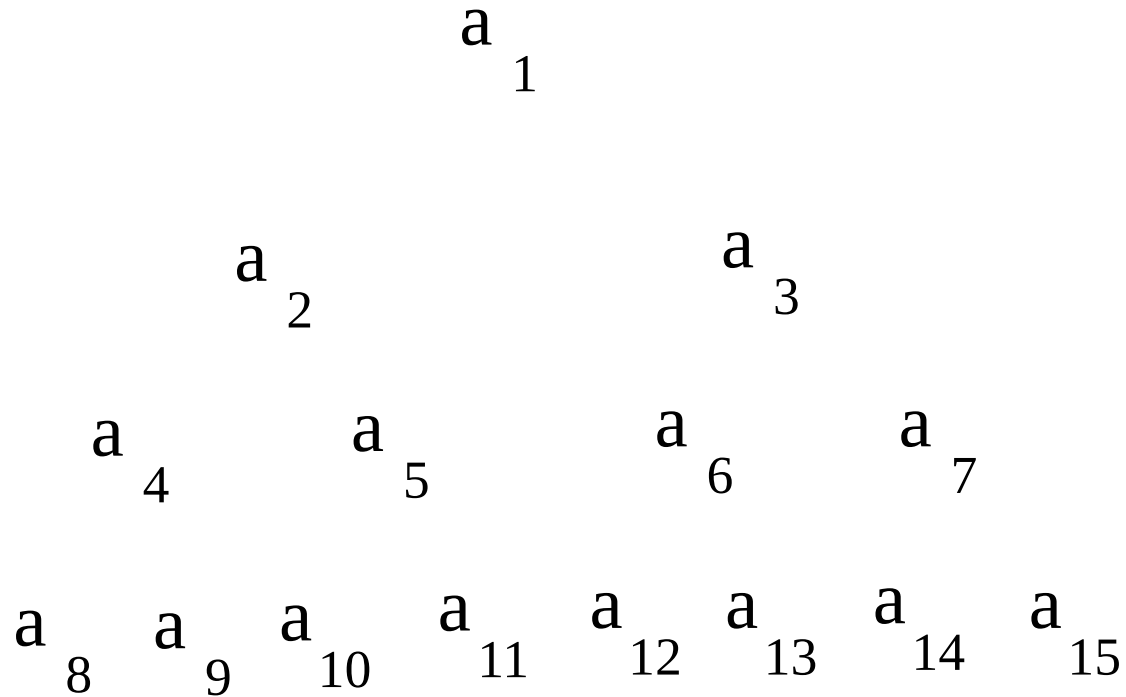
Một cách tiếp cận khác là ta sẽ làm sao lưu trữ thông tin các lần tìm kiếm trước đó để có thể sử dụng sau này.

Phương pháp Heap Sort (tt)

- Khái niệm Heap:

- Nếu có dãy các phần tử $a_1 \ a_{l+1} \ \dots \ a_r$ ($l \leq r$) mà a_i có quan hệ với $a_{2i} \ a_{2i+1}$ lớn hơn hay nhỏ hơn $\forall i = 1 \dots r$ thì ta nói đây là heap.
- Heap max là heap mà $a_i \geq \max(a_{2i}, a_{2i+1})$
- Heap min là heap mà $a_i \leq \min(a_{2i}, a_{2i+1})$
- Mọi dãy $a_1 \ a_{l+1} \ \dots \ a_r$ với ($1 \leq r < 2l$) đều là heap.

Phương pháp Heap Sort (tt)



Phương pháp Heap Sort (tt)

- Tính chất:

- Nếu có dãy $a_1 a_{l+1} \dots a_r$ là heap thì $a_i a_{i+1} \dots a_j$ ($l \leq i \leq j \leq r$) cũng là heap.
- Nếu $a_1 a_2 \dots a_k$ là heap max thì a_1 là phần tử lớn nhất.
- Nếu $a_1 a_2 \dots a_k$ là heap min thì a_1 là phần tử bé nhất.

Phương pháp Heap Sort (tt)

- Ứng dụng heap:

Nếu ta tổ chức được mảng $a_0 a_1 \dots a_{n-1}$ thành heap max thì a_0 là phần tử lớn nhất, vậy ta chỉ cần hoán đổi a_0 và a_{n-1} thì ta sẽ có a_{n-1} ở đúng vị trí. Và ta vẫn có $a_1 a_2 \dots a_{n-2}$ là một heap, vậy ta chỉ cần cập nhật thêm a_0 vào để tạo ra một heap nữa thì ta tiếp tục quá trình như trên ta sẽ được mảng có thứ tự tăng dần.

Phương pháp Heap Sort (tt)

Thuật toán:

Thuật toán HeapSort(A,n)

Input: Mảng A có n phần tử

Output: Mảng A có thứ tự

CreateHeap(A,n);

for(k $\leftarrow$ n-1 to 1) do

 A[0] $\leftrightarrow$ A[k];

 InsertHeap(A,0,k-1);

return;

Phương pháp Heap Sort (tt)

Thuật toán CreateHeap(A,n)

Input: Mảng A có n phần tử

Output: Mảng A là heap max

for($k \leftarrow (n+1)/2 - 1$ to 0) do

 InsertHeap(A,k,n-1);

return;

Phương pháp Heap Sort (tt)

Thuật toán InsertHeap(A,l,r)

Input: Mảng A chứa heap $a_{l+1} \dots a_r$

Output: Mảng A chứa heap $a_1 a_{l+1} \dots a_r$

$p \leftarrow 2 * l$;

if($p > r$) then return;

if($p < r$) then

 if($A[p] < A[p+1]$) then $p \leftarrow p+1$;

if($A[l] < A[p]$)

$A[l] \leftrightarrow A[p]$;

 InsertHeap(A,p,r);

return;

Phương pháp Heap Sort (tt)

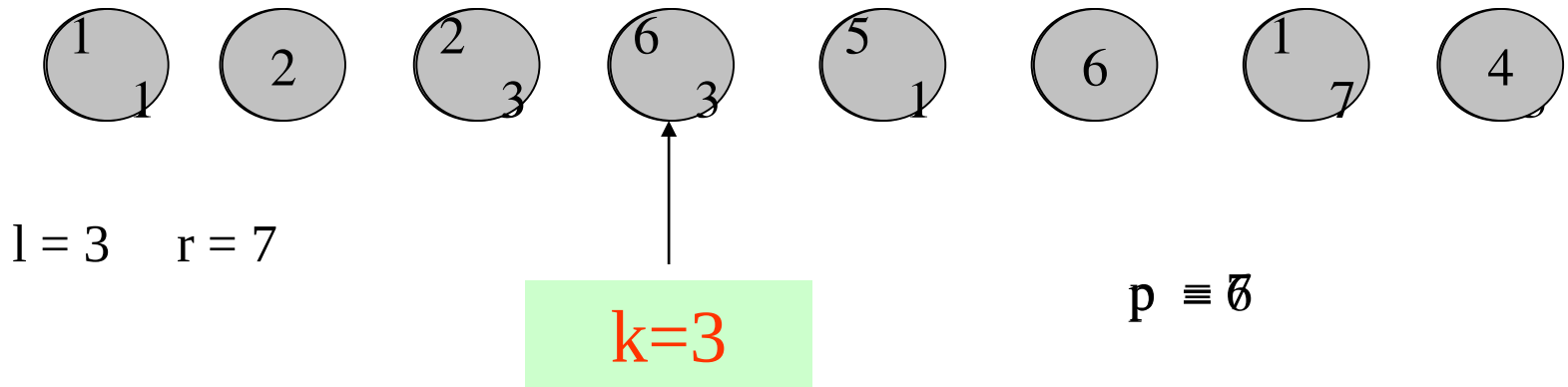
- Nhận xét:

- Thuật toán có độ phức tạp $O(n \log_2 n)$
- Muốn sắp xếp mảng tăng dùng heap max
- Muốn sắp xếp mảng giảm dùng heap min

Phương pháp Heap Sort (tt)

■ Ví dụ:

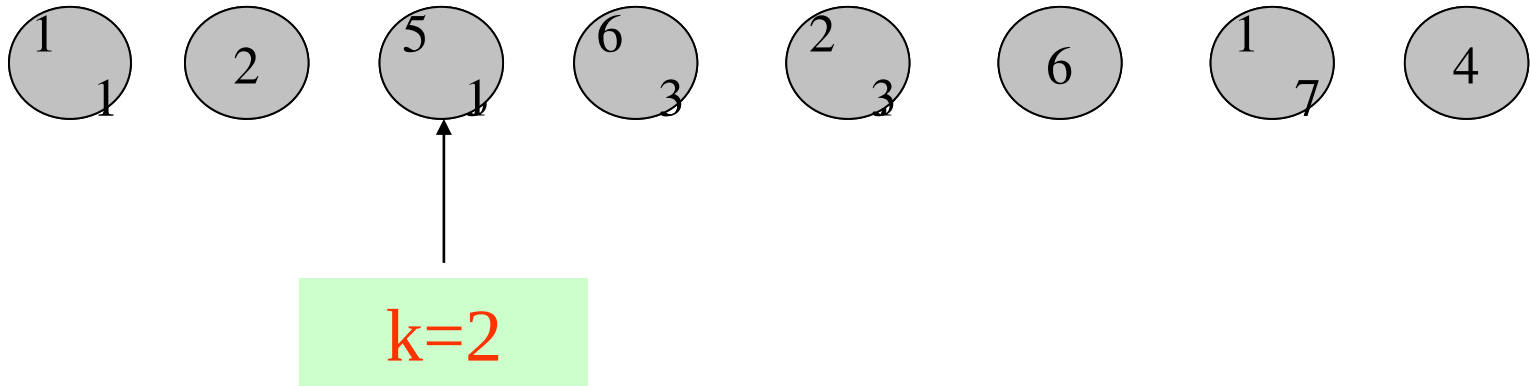
Bước 1 : Tạo Heap



Gọi InsetHeap(A,7,7) $\rightarrow$ dừng

$l = 2$ $r = 7$

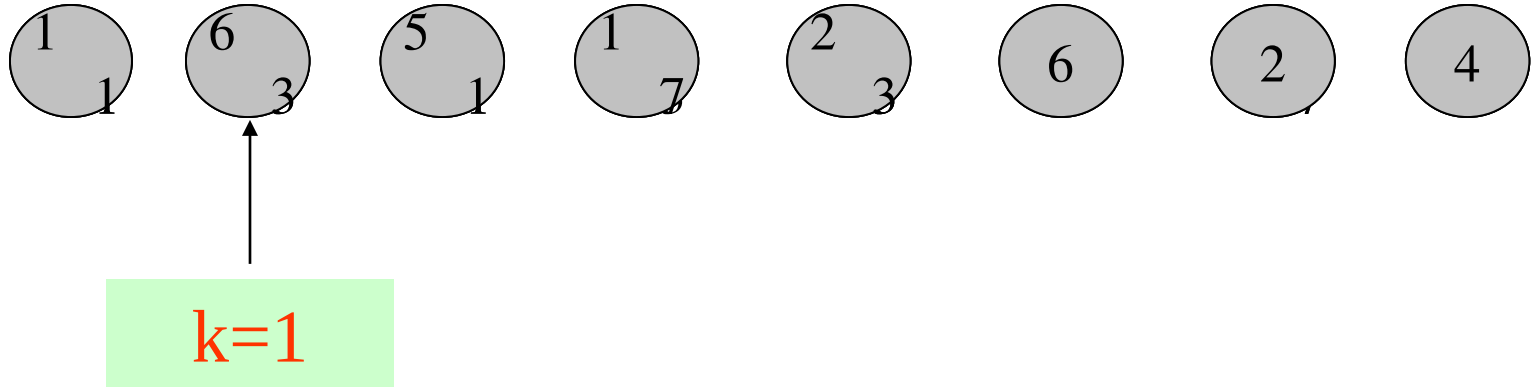
$p = 4$



Gọi $\text{InsetHeap}(A, 4, 7) \rightarrow$ dừng

$l = 1 \quad r = 7$

$p \equiv 3$



Gọi $\text{InsertHeap}(A, 6, 1, 7) \rightarrow$ dừng

$l = 0 \quad r = 7$

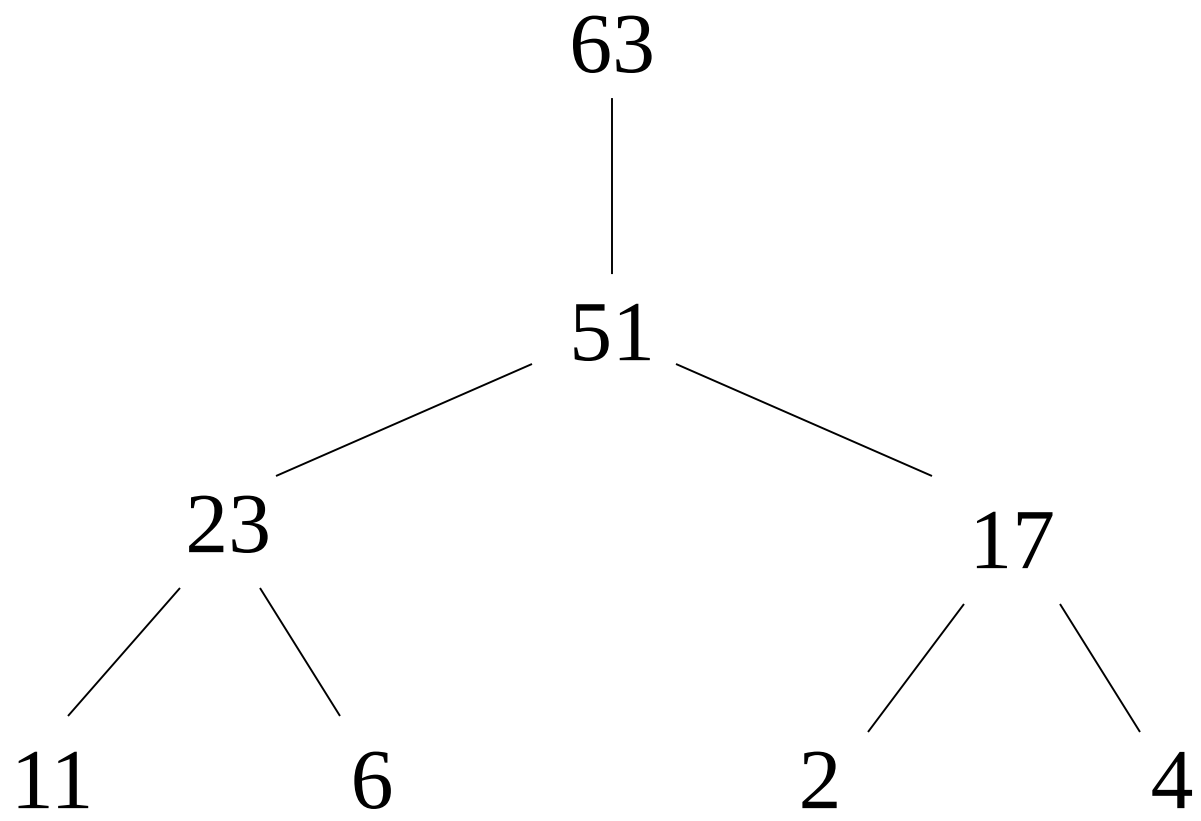
$p \equiv 0$



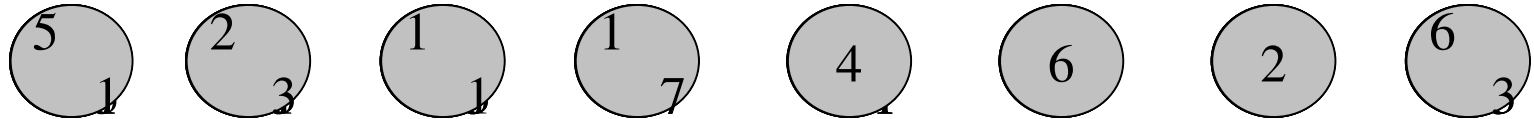
Gọi InsetHeap(A,1,7)

Gọi InsetHeap(A,2,7)

Gọi InsetHeap(A,4,7) $\rightarrow$ dừng



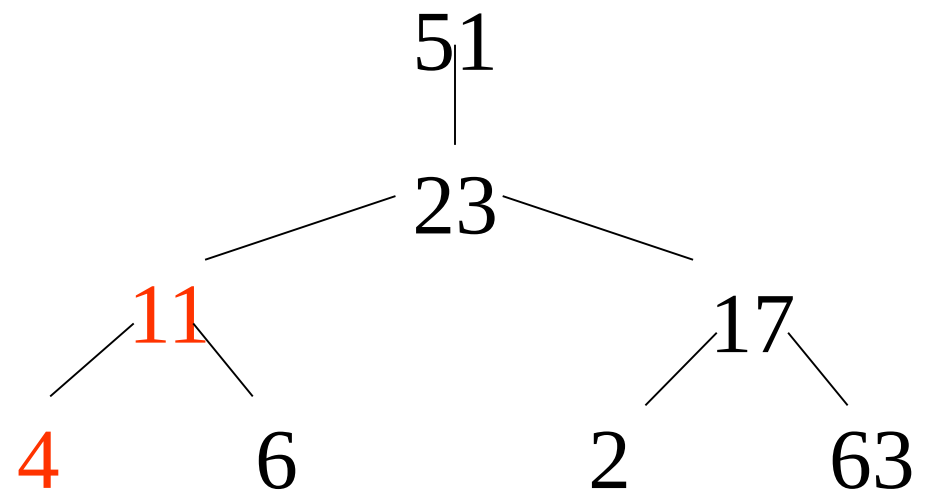
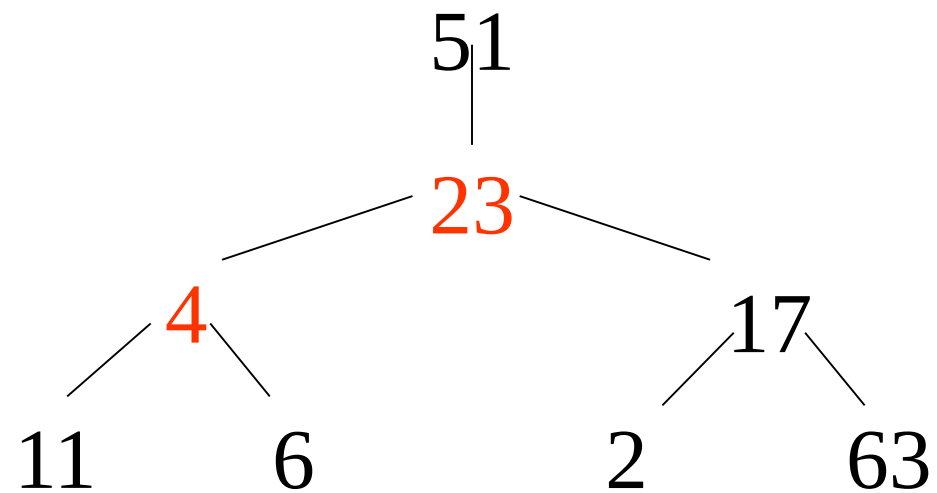
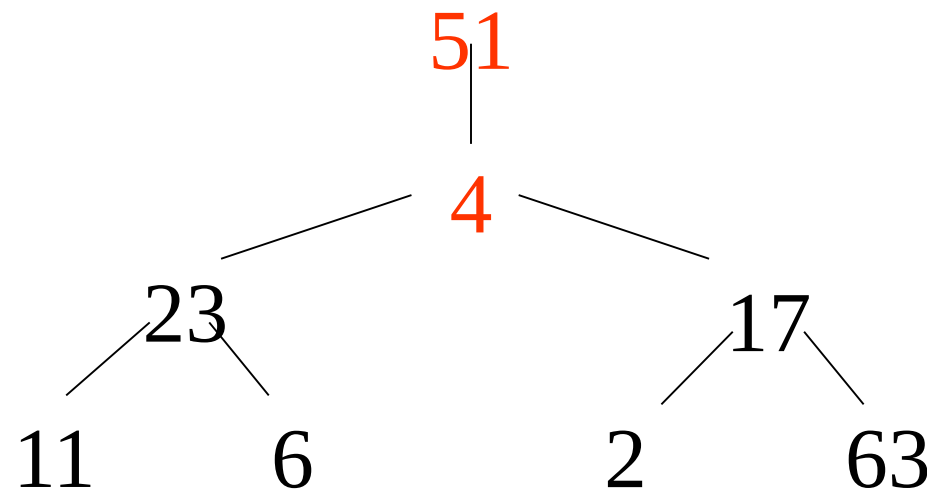
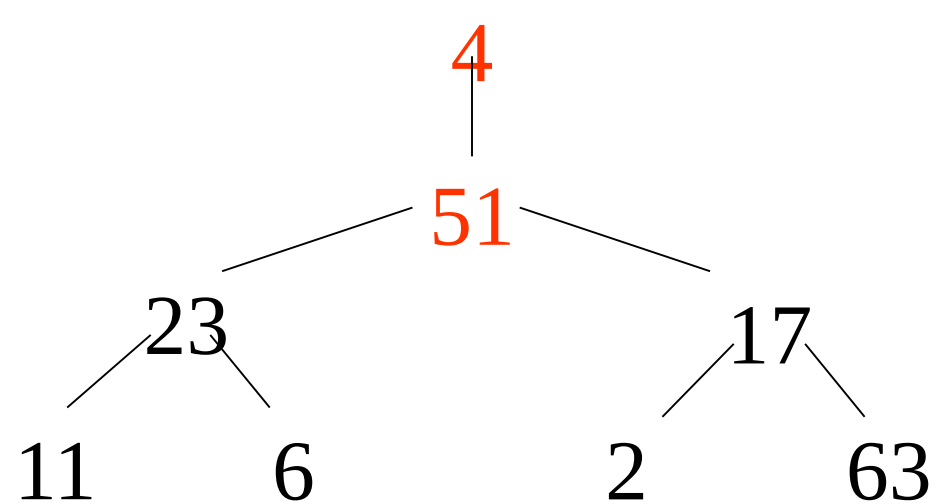
Sắp xếp



Hoán đổi 6³ và 4

Tái tổ chức Heap từ vị trí 0 đến vị trí 6

Gọi $\text{Heapify}(A, i, n)$ dùng



Phương pháp Quick Sort

- Tổng quan:

Quick Sort do Hoare đưa ra vào những năm 60, được nhiều nhà toán học, tin học nghiên cứu và phát triển như D. Knuth, N. Wirth

- Ý tưởng thuật toán:

Nguyên tắc chính là chia để trị. Đây là cơ sở của nhiều thuật toán hiệu quả.

Mảng A nếu ta chia làm 02 phần, phần đầu gồm các phần tử $\leq X$, phần sau là các phần tử $\geq X$, với X là số bất kỳ trong A. Nếu ta sắp xếp 02 phần này thì A sẽ có thứ tự.

Phương pháp Quick Sort (tt)

- Thuật toán:

Thuật toán QuickSort(A,l,r)

Input: Mảng A có các phần tử ở vị trí từ l đến r.

Output: Mảng A được sắp xếp từ l đến r.

Chọn phần tử X;

$i \leftarrow l; j \leftarrow r;$

while ($i \leq j$) do

 while ($A[i] < X$) do $i \leftarrow i + 1;$

 while ($A[j] > X$) do $j \leftarrow j - 1;$

 if ($i \leq j$) then

$A[i] \leftrightarrow A[j];$

$i \leftarrow i + 1;$

$j \leftarrow j - 1;$

 if ($j > l$) then QuickSort(A,l,j);

 if ($r > i$) then QuickSort(A,i,r);

return;

Phương pháp Quick Sort (tt)

- Nhận xét:

- Sau mỗi lần phân hoạch thì mảng A có các phần tử từ vị trí l đến j sẽ có giá trị $\leq X$ và từ i đến r sẽ $\geq X$.
- Khi phân chia bắt buộc phần tử X phải là phần tử trong mảng để đảm bảo tính dừng của thuật toán (cả hai phần sau khi chia sẽ không bị rỗng).
- Trung bình thuật toán có độ phức tạp $O(n \log_2 n)$

Phương pháp Quick Sort (tt)

- Trong trường hợp xấu nhất là $O(n^2)$
- Trường hợp xấu nhất là sau khi phân hoạch thì một phần chứa 01 phần tử và phần kia chứa $l-r$ phần tử còn lại. Trường hợp tốt nhất xảy ra khi chia mảng được 02 phần xấp xỉ gần bằng nhau về số lượng (chỉ sai khác 01 phần tử). Do đó, trong cài đặt lập trình viên thường chọn phần tử $X = A[(l+r)/2]$ với hy vọng sẽ là phần tử trung vị (median)

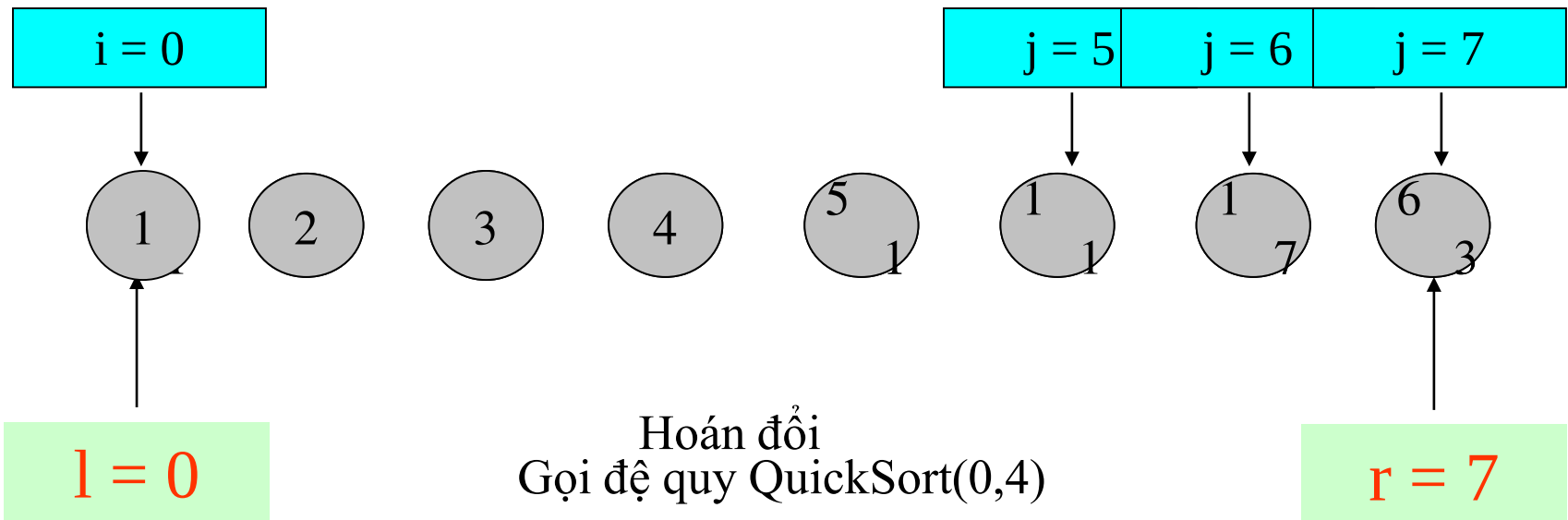
Phương pháp Quick Sort (tt)

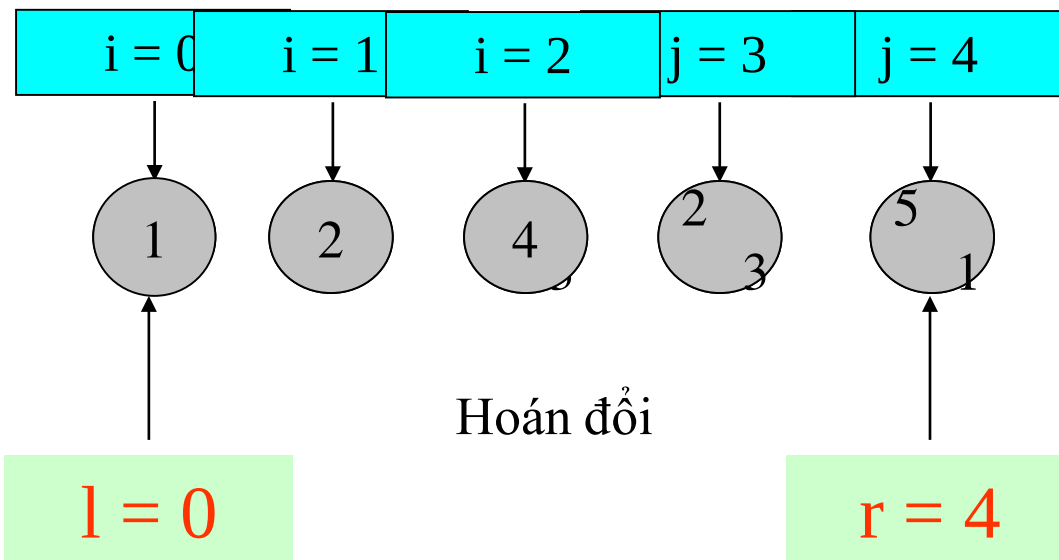
- Trong nhiều cải tiến, người ta cố gắng chọn phần tử X không phải là phần tử lớn nhất hay nhỏ nhất.
- Cuối cùng, khi n nhỏ, lập trình viên sẽ không dùng quick sort mà sẽ dùng thuật toán khác hiệu quả hơn như: chọn trực tiếp, nổi bọt, Nhưng khi n khá lớn chúng ta sẽ gặp khó khăn với vấn đề đệ quy.

Phương pháp Quick Sort (tt)

- Ví dụ:

Chọn $X = A[(7+0)/2] = 4$

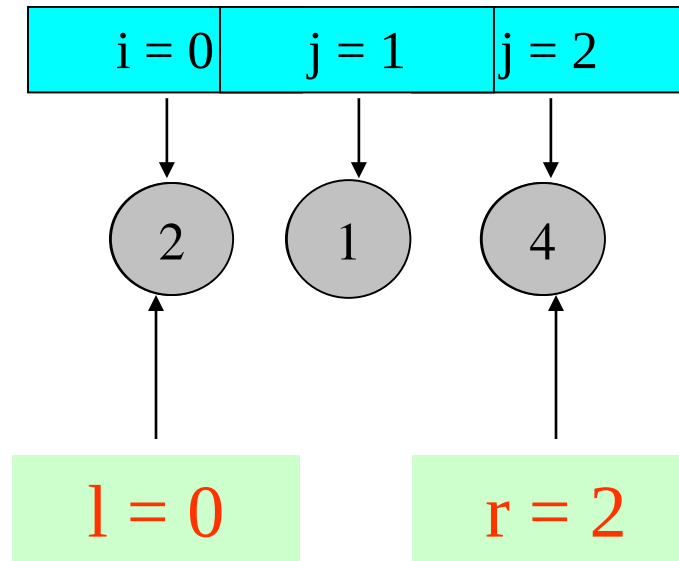




Chọn $X = A[(4+0)/2] = 23$

Gọi đệ quy QuickSort(0,2)

Hoán đổi

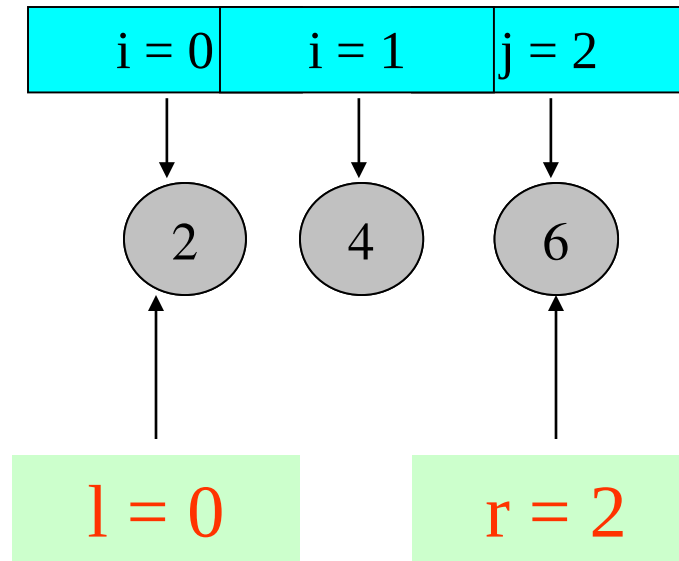


Chọn $X = A[(2+0)/2] = 2$

Gọi đệ quy QuickSort(0,1) $\rightarrow$ dừng

Gọi đệ quy QuickSort(0,2) nhánh if($r > i$)

Hoán đổi



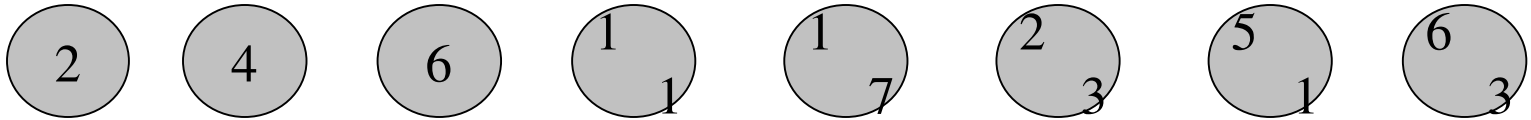
Chọn $X = A[(2+0)/2] = 6$

Gọi đệ quy QuickSort(0,2) $\rightarrow$ dừng

Gọi đệ quy QuickSort(1,2) $\rightarrow$ dừng

Gọi đệ quy QuickSort nhánh còn lại

Cuối cùng ta được



Phương pháp Merge Sort

- Tổng quan:

Merge Sort do D. Knuth giới thiệu vào những năm 60

- Ý tưởng thuật toán:

Nguyên tắc chính cũng là chia để trị. Nhưng cách chia sẽ khác Quick Sort.

Mảng A nếu ta chia làm 02 phần bằng nhau, nếu ta sắp xếp 02 phần lại rồi trộn 02 nửa lại ta sẽ có mảng có thứ tự.

Phương pháp Merge Sort (tt)

- Thuật toán:

Thuật toán MergeSort(A,l,r)

Input: Mảng A có các phần tử ở vị trí từ 1 đến r.

Output: Mảng A được sắp xếp từ 1 đến r.

if($l < r$) then

$mid \leftarrow (l+r)/2$;

 MergeSort(A,l,mid);

 MergeSort(A,mid+1,r)

 Trộn A[l,mid] và A[mid+1,r] thành A[l,r];

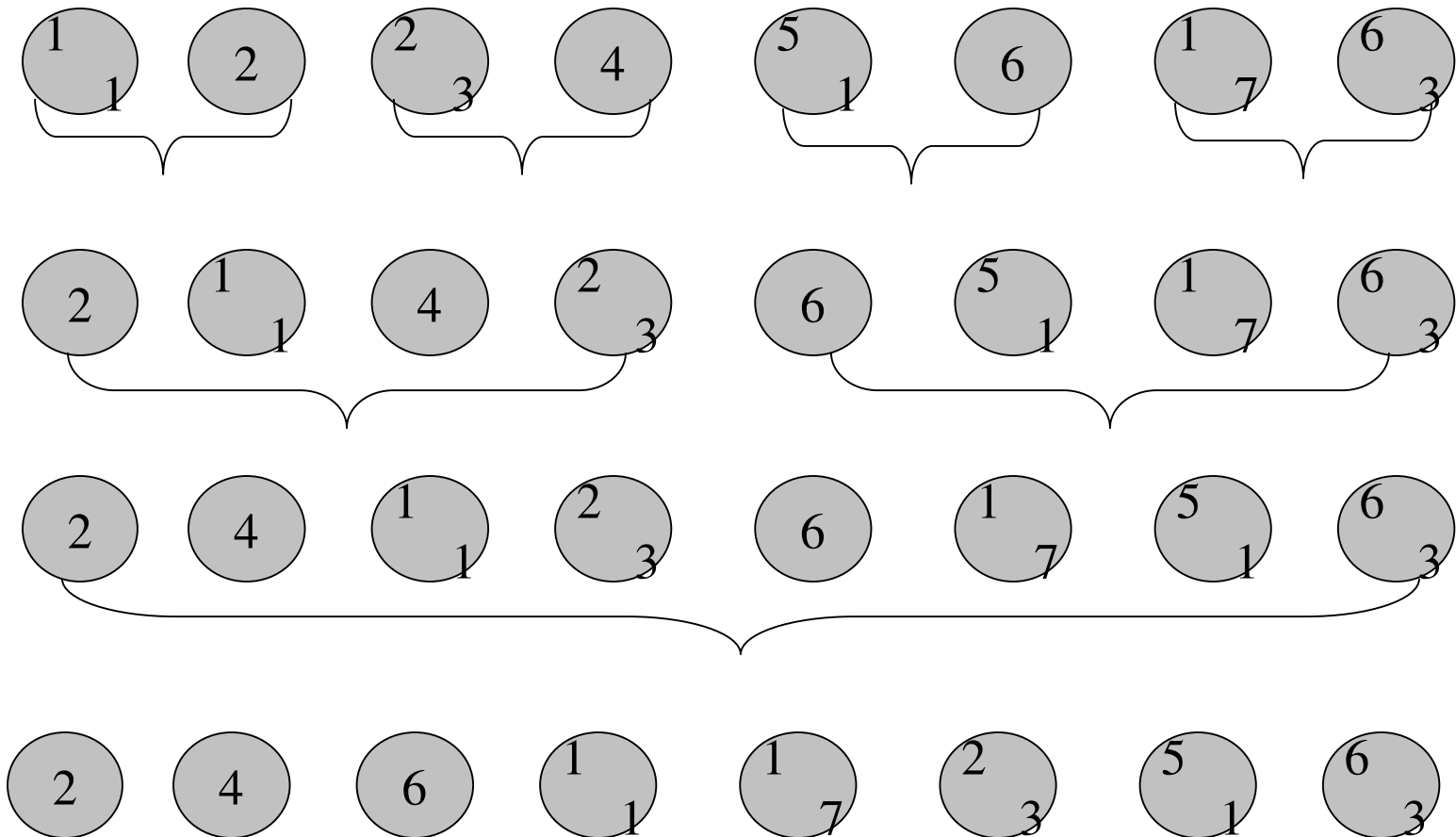
return;

Phương pháp Merge Sort (tt)

- Nhận xét:
 - Sau mỗi lần phân hoạch thì số lượng sắp xếp sẽ giảm $\frac{1}{2}$, vậy cần $\log_2 n$ lần phân hoạch.
 - Chi phí trung bình là $O(n \log_2 n)$

Phương pháp Merge Sort (tt)

■ Ví dụ:



Phương pháp Merge Sort (tt)

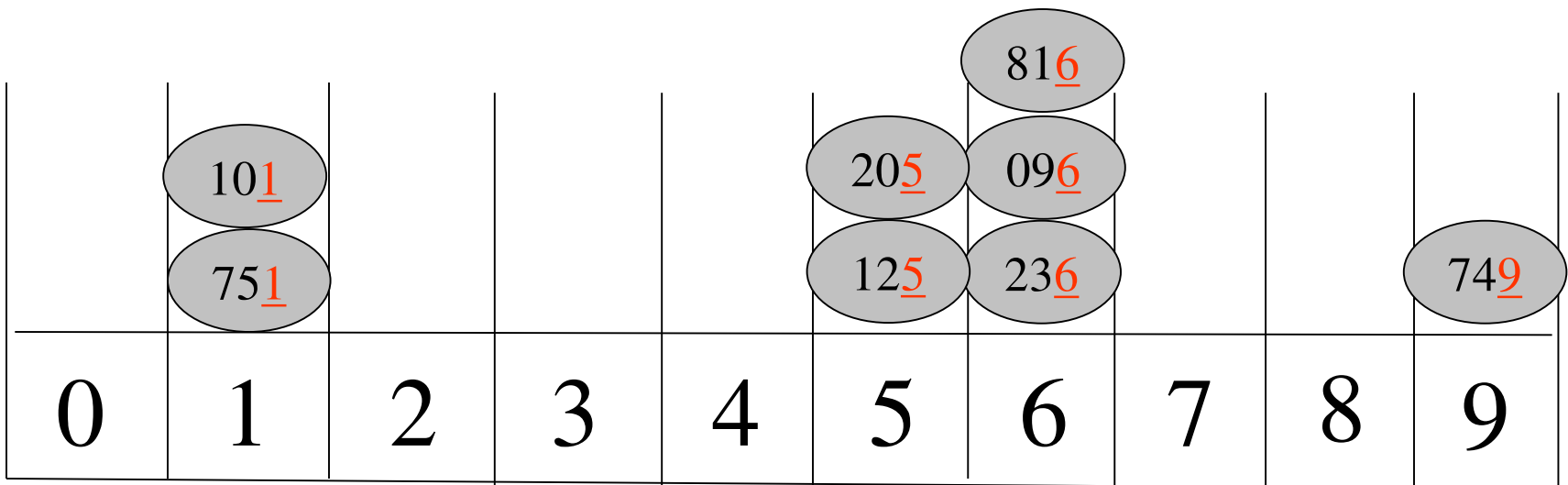
- Cải tiến:
 - Trộn k dãy
 - Trộn dãy có thứ tự
 - ...

Phương pháp Radix Sort

- Ý tưởng thuật toán:
 - Radix Sort là thuật toán tiếp cận theo ý tưởng không so sánh giá trị các phần tử; cơ sở để sắp xếp của các thuật toán cũ; mà dùng nguyên tắc phân loại thư như bưu điện. Do đó thuật toán này còn có tên là Postman's Sort.
 - Nguyên tắc chính là phân loại và trình tự phân loại sẽ tạo ra thứ tự cho các phần tử.

Phương pháp Radix Sort (tt)

- Ví dụ:



751

101

125

205

236

096

816

749

0	1	2	3	4	5	6	7	8	9

751 101 125 205 236 096 816 749

0	1	2	3	4	5	6	7	8	9

101

205

816

125

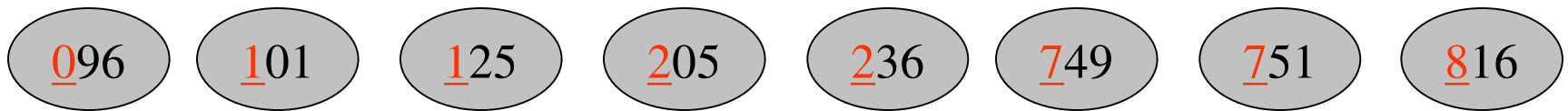
236

749

751

096

0	1	2	3	4	5	6	7	8	9



Kết thúc → Mảng có thứ tự

Phương pháp Radix Sort (tt)

- Thuật toán:

Thuật toán RadixSort(A, n, k)

Input: Mảng A có n phần tử, có tối đa k chữ số.

Output: Mảng A được sắp xếp.

Khởi tạo 10 ngăn chứa $B[0..9]$ rỗng;

for ($t \leftarrow 0$ to $k-1$) do

 for ($j \leftarrow 0$ to $n-1$) do

 Thêm $A[j]$ vào $B[\text{Digit}(A[j], t)]$;

 for ($h \leftarrow 0$ to 9) do

 Lấy ngược các phần tử từ $B[h]$ đưa vào A

return;

Phương pháp Radix Sort (tt)

Thuật toán Digit(n,k)

Input: số nguyên n và k.

Output: giá trị tại vị trí k của số n.

value $\leftarrow$ 1;

for (i $\leftarrow$ 0 to k-1) do

 value $\leftarrow$ value * 10;

return (n / value)% 10;

Phương pháp Radix Sort (tt)

- Nhận xét:

- Độ phức tạp của thuật toán là $O(n)$.
- Tốn kém, vì phải có 10 ngăn chứa mà mỗi ngăn chứa phải có tối đa đúng bằng số phần tử của dữ liệu với kiểu dữ liệu số, và nhiều hơn nữa với kiểu dữ liệu chuỗi.
- Thuận tiện cài đặt với các kiểu dữ liệu chỉ so sánh trên số, ký tự, chuỗi.
- Rất thích hợp cho sắp xếp trên cấu trúc dữ liệu danh sách liên kết

Thuật toán sắp xếp nào là nhanh nhất?

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
<u>Quicksort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(\log(n))$
<u>Mergesort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Timsort</u>	$\Omega(n)$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Heapsort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(1)$
<u>Bubble Sort</u>	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Insertion Sort</u>	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Selection Sort</u>	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Tree Sort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(n)$
<u>Shell Sort</u>	$\Omega(n \log(n))$	$\Theta(n(\log(n))^2)$	$O(n(\log(n))^2)$	$O(1)$
<u>Bucket Sort</u>	$\Omega(n+k)$	$\Theta(n+k)$	$O(n^2)$	$O(n)$
<u>Radix Sort</u>	$\Omega(nk)$	$\Theta(nk)$	$O(nk)$	$O(n+k)$
<u>Counting Sort</u>	$\Omega(n+k)$	$\Theta(n+k)$	$O(n+k)$	$O(k)$
<u>Cubesort</u>	$\Omega(n)$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$

<https://techtalk.vn/thuat-toan-sap-xep-nao-la-nhanh-nhat.html>

Machine Learning and Sort

An $O(N)$ Sorting Algorithm: Machine Learning Sorting

Hanqing Zhao and Yuehan Luo

Department of Modern Physics, University of Science and Technology of China, Hefei 230026, China

School of Physical Science and Technology, and Key Laboratory for Magnetism and Magnetic Materials of MOE, Lanzhou University, Lanzhou, Gansu 730000, China

Department of Applied Mathematics and Statistics, State University of New York at Stony Brook, Stony Brook, NY, 11794, USA

(2018)

“Lots of classic algorithms have been designed and applied, such as Bubble Sort, Selection Sort, Insertion Sort, etc. However, it’s been proven that sorting algorithms based on comparison have a fundamental requirement of $\Omega(N \log N)$ comparisons[2, 3], which implies the time complexity is at least $O(N \log N)$ [4–6]. The non-comparison sorting algorithms, such as Bucket Sort, Counting Sort and Radix Sort, are not restricted by the $\Omega(N \log N)$ boundary, and can reach $O(N)$ complexity [7], but these algorithms have very limited applications. Most of the state-of-art sorting algorithms employ parallel computing to handle big datasets and have accomplished outstanding achievements [8–12]. For example [13], in 2015, FuxiSort [14], developed by Alibaba Group, is a distributed sort implementation on top of Apsara. FuxiSort is able to complete the **100TB Daytona GraySort benchmark in 377 seconds** on random nonskewed dataset and 510 seconds on skewed dataset, and Indy GraySort benchmark in 329 seconds. Then, in 2016, Tencent Sort [15] has achieved a speed of 60.7 TB/min in sorting 100 TB data for the Indy GraySort, using a cluster of 512 OpenPOWER servers optimized for hyperscale data centers. **However, these algorithms are still limited by the lower boundary complexity of comparison sorting algorithm and time-consuming networking** [16].”